FASTAPI & FLASK

CHEAT SHEETS

A Complete Reference for FastAPI & Flask Web APIs

Path Operations · Pydantic · Dependencies · Responses · SQLModel · OAuth2 & JWT · Async · WebSockets · Testing · Deployment · Flask · Jinja2

60 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & the ASGI Stack
02 Installation & the fastapi CLI
03 First App & Project Layout
04 Interactive Docs & OpenAPI

PATHS & PARAMETERS

05 Path Operations
06 Path Parameters
07 Query Parameters
08 Request Body
09 Validation with Query, Path & Annotated
10 Headers, Cookies, Forms & Files

PYDANTIC MODELS

11 Pydantic BaseModel
12 Field & Constraints
13 Custom Validators
14 Nested Models & Special Types
15 Response Models & Filtering
16 model_config & Settings

RESPONSES & ERRORS

17 Response Types
18 Status Codes, Headers & Cookies
19 HTTPException & Exception Handlers
20 Validation Errors & Error Formats

DEPENDENCY INJECTION

21 Depends Basics
22 Class & Sub-Dependencies
23 Dependencies with yield
24 Global Dependencies & Overrides

APP STRUCTURE

25 APIRouter & Bigger Applications
26 Lifespan Events
27 Middleware & CORS

DATABASES

28 SQLModel
29 SQLAlchemy 2.0
30 Async Database Access
31 Alembic Migrations & CRUD Patterns

SECURITY

32 Security Basics & OAuth2 Password Flow
33 JWT Authentication
34 Password Hashing & Users
35 Scopes, API Keys & HTTP Basic

ASYNC & REAL-TIME

36 async def vs def
37 Background Tasks
38 WebSockets
39 Streaming & Server-Sent Events

TESTING

FLASK

46 Flask Introduction & First App
47 Routing & URL Building
48 Request & Response
49 Templates with Jinja2
50 Forms & Flask-WTF
51 Blueprints & the App Factory
52 Contexts, g & Configuration
53 Request Hooks & Error Handling
54 Flask-SQLAlchemy & Flask-Migrate
55 Sessions & Flask-Login
56 REST APIs & Async in Flask
57 Testing & Deploying Flask

CHOOSING A FRAMEWORK

58 FastAPI vs Flask vs Django

QUICK REFERENCE

59 Quick Reference — FastAPI
60 Quick Reference — Flask

Table of Contents

60 reference sheets · syntax · examples · notes

Sheet 01
Introduction & the ASGI Stack
Starlette · Pydantic · Uvicorn · ASGI · type hints · OpenAPI

Sheet 02
Installation & the fastapi CLI
pip install fastapi[standard] · uv add · venv · fastapi dev · fastapi run · uvicorn

Sheet 03
First App & Project Layout
app = FastAPI() · @app.get · title · version · app/main.py · __init__.py

Sheet 04
Interactive Docs & OpenAPI
/docs · /redoc · /openapi.json · summary · description · tags · docstrings

Sheet 05
Path Operations
@app.get · post · put · patch · delete · route order · status_code · operation_id

Sheet 06
Path Parameters
/items/{item_id} · type conversion · Enum · {file_path:path} · 422 errors

Sheet 07
Query Parameters
non-path args · defaults · str | None = None · bool parsing · list[str] · Query()

Sheet 08
Request Body
Pydantic model param · JSON body · body + path + query · Body(embed=True) · multiple bodies

Sheet 09
Validation with Query, Path & Annotated
Annotated[int, Path(ge=1)] · Query(max_length=50) · pattern= · alias= · deprecated=True

Sheet 10
Headers, Cookies, Forms & Files
Header() · Cookie() · Form() · File() · UploadFile · python-multipart

Sheet 11
Pydantic BaseModel
class Item(BaseModel) · type coercion · defaults · model_dump() · model_validate() · model_dump_json()

Sheet 12
Field & Constraints
Field(gt=0, max_length=…) · default_factory · examples= · EmailStr · HttpUrl · Annotated

Sheet 13
Custom Validators
@field_validator · @model_validator · mode=before/after · AfterValidator · @computed_field

Sheet 14
Nested Models & Special Types
nested models · list[Item] · dict[str, int] · Literal · datetime / UUID / Decimal · discriminated unions

Sheet 15
Response Models & Filtering
response_model= · -> return type · ItemIn / ItemOut · response_model_exclude_unset · from_attributes

Sheet 16
model_config & Settings
ConfigDict · extra=forbid · populate_by_name · pydantic-settings · BaseSettings · .env · lru_cache

Sheet 17
Response Types
JSONResponse · HTMLResponse · PlainTextResponse · RedirectResponse · FileResponse · StreamingResponse

Sheet 18
Status Codes, Headers & Cookies
status.HTTP_201_CREATED · Response parameter · response.headers · set_cookie() · responses=

Sheet 19
HTTPException & Exception Handlers
raise HTTPException(404) · detail= · headers= · @app.exception_handler · custom exceptions

Sheet 20
Validation Errors & Error Formats
422 shape · RequestValidationError · override handler · jsonable_encoder · error envelopes

Sheet 21
Depends Basics
Depends() · Annotated[T, Depends(fn)] · shared params · type aliases · per-request cache

Sheet 22
Class & Sub-Dependencies
classes as dependencies · __call__ · dependency chains · use_cache=False · parameterized

Sheet 23
Dependencies with yield
yield setup/teardown · DB session per request · try / finally · exceptions after yield · cleanup order

Sheet 24
Global Dependencies & Overrides
FastAPI(dependencies=[…]) · APIRouter(dependencies=…) · decorator dependencies= · dependency_overrides

Sheet 25
APIRouter & Bigger Applications
APIRouter(prefix=, tags=) · include_router() · routers/ folder · shared deps · relative imports · /v1

Sheet 26
Lifespan Events
@asynccontextmanager · lifespan= · startup/shutdown · app.state · on_event (deprecated)

Sheet 27
Middleware & CORS
@app.middleware("http") · call_next · CORSMiddleware · allow_origins · GZipMiddleware · TrustedHostMiddleware

Sheet 28

Sheet 29

Sheet 30

Introduction & the ASGI Stack

DESCRIPTION

FastAPI is a Python web framework for building APIs from ordinary type hints. You declare what a request should look like, and FastAPI parses it, validates it, converts it, and documents it in OpenAPI. It sits on **Starlette** (the web toolkit) and **Pydantic** (validation), and runs on an **ASGI** server such as **Uvicorn**.

NOTES

- Created by Sebastián Ramírez, first released in 2018
- Still versioned `0.x` : pin the version in production
- Needs Python 3.10+ for current releases (`X | None` syntax)
- Async-first, but plain `def` endpoints work too

UNDER THE HOOD

- Starlette provides routing, requests, responses, middleware and WebSockets
- Pydantic v2 (Rust core) does parsing and validation
- Each request becomes an ASGI `scope` + `receive` / `send` call

HELLO, FASTAPI — MAIN.PY

```python
from fastapi import FastAPI

app = FastAPI(title="Todo API")       # the ASGI application

@app.get("/")                          # GET /
def root():
    return {"message": "Hello, FastAPI"}   # dict → JSON

@app.get("/todos/{todo_id}")
def read_todo(todo_id: int, done: bool = False):
    return {"id": todo_id, "done": done}

# $ fastapi dev main.py
# GET /todos/3?done=true → {"id": 3, "done": true}
# GET /todos/abc         → 422 (not an int)
# Docs: http://127.0.0.1:8000/docs
```

CODE EXPLAINED

- → `FastAPI()` — the app object; an ASGI callable
- → `@app.get(path)` — registers a path operation
- → `todo_id: int` — path param, converted and validated
- → `done: bool = False` — not in the path, so it is a query param
- → Return value is serialized to JSON automatically

OTHER FRAMEWORK

- Flask — `@app.get("/todos/<int:todo_id>")` , `request.args`
- Django — `path("todos/<int:id>/", view)` in `urls.py`

PREFER / AVOID

- Avoid parsing `request.query_params` by hand · Prefer typed function parameters
- Avoid returning `json.dumps(...)` · Prefer returning dicts or models

WSGI VS ASGI

	WSGI	ASGI
Frameworks	Flask, Django (classic)	FastAPI, Starlette, Django async
Concurrency	one request per worker thread	async event loop
WebSockets	no	yes
Servers	Gunicorn, Waitress	Uvicorn, Hypercorn, Granian

WHAT FASTAPI GENERATES

FEATURE	FROM
Validation	type hints + Pydantic
Docs	OpenAPI 3.1 schema
Editor help	real Python types
Error responses	422 with details

⚠ COMMON MISTAKES

- Running `python main.py` without a server: FastAPI needs Uvicorn or `fastapi dev`
- Expecting `async def` to speed up blocking code: it does not (see sheet 36)
- Leaving the version unpinned; minor releases can change behavior

✓ TIPS & BEST PRACTICES

- Open `/docs` first: it shows exactly how FastAPI read your hints
- Start with `fastapi dev` ; it reloads on save
- Read Starlette docs for anything request/response-level

QUICK SUMMARY

- `FastAPI()` — ASGI app built on Starlette + Pydantic
- `@app.get()` — path operation decorator
- type hints — drive parsing, validation and docs
- `/docs` — free interactive API docs

Installation & the fastapi CLI

DESCRIPTION

Install FastAPI into a **virtual environment** with the `standard` extra, which brings Uvicorn, the `fastapi` CLI, `httpx`, `jinja2`, `python-multipart` and email validation. The CLI gives you `fastapi dev` for development (auto-reload, localhost only) and `fastapi run` for production (no reload, listens on all interfaces).

NOTES

- `fastapi` alone has no server; use `fastapi[standard]`
- Quote the extra in zsh: `"fastapi[standard]"`
- `fastapi dev` finds `main.py`, `app.py` or `api.py` automatically
- `uv` is a fast pip + venv replacement; both work

SET UP A PROJECT

```
# pip + venv
python3 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install "fastapi[standard]"
pip freeze > requirements.txt

# or uv
uv init todo-api && cd todo-api
uv add "fastapi[standard]"

# check it
python -c "import fastapi; print(fastapi.__version__)"

# run
fastapi dev main.py                  # reload, 127.0.0.1:8000
fastapi run main.py --port 8080      # prod, 0.0.0.0, no reload
uvicorn main:app --reload            # same thing, by hand
```

CODE EXPLAINED

- → `python3 -m venv .venv` — isolated environment per project
- → `[standard]` — server, CLI and common optional deps
- → `uv add` — installs and records in `pyproject.toml` + `uv.lock`
- → `fastapi dev` — development server with reload
- → `main:app` — module `main`, variable `app`

OTHER FRAMEWORK

- Flask — `pip install flask`, run with `flask --app app run --debug`
- Django — `pip install django`, run with `manage.py runserver`

PREFER / AVOID

- Avoid installing into the system Python · Prefer a venv or `uv`
- Avoid `--reload` in production · Prefer `fastapi run --workers N`

FASTAPI CLI OPTIONS

OPTION	DOES
`--host 0.0.0.0`	listen on all interfaces
`--port 8080`	change port (default 8000)
`--reload / --no-reload`	toggle reload
`--workers 4`	multiple processes (run)
`--app app`	pick the app variable
`--root-path /api`	behind a proxy prefix

COMMON EXTRAS

PACKAGE	WHY
`uvicorn[standard]`	ASGI server + uvloop
`python-multipart`	forms & file uploads
`httpx`	TestClient + async HTTP
`pydantic-settings`	env-based config
`sqlmodel`	ORM (sheet 28)

⚠ COMMON MISTAKES

- `pip install fastapi[standard]` in zsh fails: quote the brackets
- Using `fastapi dev` in Docker: it only listens on 127.0.0.1
- Forgetting to activate the venv, then `fastapi: command not found`

✓ TIPS & BEST PRACTICES

- Commit `uv.lock` or a pinned `requirements.txt` for repeatable builds
- Use `fastapi run` in containers; it binds `0.0.0.0` already
- `uv run fastapi dev` runs inside the project env without activating

QUICK SUMMARY

- `fastapi[standard]` — framework + server + CLI
- `fastapi dev` — reload, localhost
- `fastapi run` — production, all interfaces
- `uvicorn main:app` — the server underneath

First App & Project Layout

DESCRIPTION

A FastAPI app is one `FastAPI()` object plus functions decorated with HTTP methods. Small projects fit in `main.py`; anything bigger moves into an **app/ package** with routers, models and settings split into modules. Pass **metadata** to `FastAPI()` so the docs show a real title, version and description.

NOTES

- Return dicts, lists, Pydantic models or `Response` objects
- Functions run in order of registration when paths overlap
- `app/__init__.py` makes `app` an importable package
- Run a package app with `fastapi dev app/main.py`

APP/MAIN.PY

```python
from fastapi import FastAPI
from app.routers import todos, users      # split by feature

app = FastAPI(
    title="Todo API",
    version="1.2.0",
    description="Tracks **todos** for each user.",
    contact={"name": "Bob", "url": "https://example.com"},
)

app.include_router(todos.router)          # sheet 25
app.include_router(users.router)

@app.get("/health", tags=["ops"])
def health():
    return {"status": "ok"}
```

CODE EXPLAINED

→ `title` / `version` / `description` — shown at the top of `/docs`

→ `description` accepts Markdown

→ `include_router` — mounts routes from another module

→ `tags=[...]` — groups operations in the docs

→ `/health` — cheap endpoint for load balancers

OTHER FRAMEWORK

- Flask — `create_app()` factory + blueprints (sheet 51)
- Django — one project with many apps (`startapp`)

PREFER / AVOID

- Avoid one 2,000-line `main.py` · Prefer routers per feature
- Avoid circular imports via `from main import app` · Prefer routers that never import `app`

TYPICAL LAYOUT

PATH	HOLDS
app/main.py	FastAPI(), include_router
app/routers/	one module per resource
app/models.py	DB tables
app/schemas.py	Pydantic in/out models
app/deps.py	shared dependencies
app/config.py	settings (sheet 16)
tests/	pytest + TestClient

FASTAPI() ARGUMENTS

ARG	PURPOSE
title, version	docs header
summary, description	docs text
openapi_tags	tag descriptions
docs_url, redoc_url	move or disable docs
lifespan	startup/shutdown (26)
dependencies	global deps (24)

⚠ COMMON MISTAKES

- Naming the file `fastapi.py` : it shadows the real package
- Importing `app` inside routers, causing circular imports
- Defining `/users/me` after `/users/{id}` : the param route wins

✓ TIPS & BEST PRACTICES

- Keep schemas (API shapes) separate from DB models
- Add `/health` early; deploy tools expect it
- Use `tags` on routers so `/docs` stays organized

QUICK SUMMARY

- `FastAPI(title=, version=)` — app + docs metadata
- `app/` package — routers, schemas, models, deps
- `include_router()` — compose the app
- registration order — decides overlapping matches

Interactive Docs & OpenAPI

DESCRIPTION

FastAPI builds an **OpenAPI** schema from your routes, parameters and models, then serves two UIs from it: **Swagger UI** at `/docs` (try requests in the browser) and **ReDoc** at `/redoc` (read-only reference). The raw schema at `/openapi.json` can generate clients, import into Postman, or drive contract tests.

NOTES

- Docstrings become operation descriptions (Markdown allowed)
- `summary` overrides the auto title made from the function name
- `response_description` labels the success response
- Disable in prod with `docs_url=None, redoc_url=None`

DOCUMENTING AN OPERATION

```python
@app.post(
    "/todos",
    tags=["todos"],
    summary="Create a todo",
    response_description="The created todo",
    status_code=201,
)
def create_todo(todo: TodoIn) -> TodoOut:
    """
    Create a todo for the current user.

    - **title**: required, 1-100 chars
    - **due**: optional ISO date
    """
    ...

@app.get("/legacy", deprecated=True, include_in_schema=True)
def legacy(): ...
```

CODE EXPLAINED

- → `tags` — section heading in Swagger UI
- → `summary` — one-line title for the operation
- → docstring — long description, rendered as Markdown
- → `-> TodoOut` — documents the response schema
- → `deprecated=True` — shown struck through in the docs

OTHER FRAMEWORK

- Flask — no built-in docs; use `flask-smorest` or `APIFlask` (sheet 56)
- Django — DRF + `drf-spectacular` for OpenAPI

PREFER / AVOID

- Avoid writing a separate API doc by hand · Prefer docstrings + models
- Avoid exposing `/docs` on a public prod API unintentionally · Prefer an explicit choice

DOC URLS

URL	SHOWS
`/docs`	Swagger UI, try it out
`/redoc`	ReDoc reference
`/openapi.json`	raw schema (JSON)
`/docs/oauth2-redirect`	OAuth2 login helper

OPERATION DOC PARAMS

PARAM	EFFECT
`tags=[...]`	group operations
`summary=`	short title
`description=`	long text (or docstring)
`response_description=`	success label
`deprecated=True`	mark as deprecated
`include_in_schema=False`	hide from docs

⚠ COMMON MISTAKES

- Expecting `/docs` to work offline: Swagger UI loads assets from a CDN by default
- Using `print`-style docstrings with odd indentation: Markdown renders them as code
- Forgetting response models, so docs show an empty `{}` response

✓ TIPS & BEST PRACTICES

- Use `openapi_tags` (sheet 43) to add descriptions to each tag
- Import `/openapi.json` into Postman or Bruno to get a ready collection
- Add `examples` to models (sheet 12) so "Try it out" is pre-filled

QUICK SUMMARY

- `/docs` — Swagger UI
- `/redoc` — ReDoc
- `/openapi.json` — schema for tools
- docstring + `summary` — operation text

Path Operations

DESCRIPTION

A **path operation** is one HTTP method on one path, handled by one function. FastAPI has a decorator per method. The decorator holds routing and docs settings (status code, tags, response model); the function signature holds the inputs. Routes are matched **in the order they were added**, so fixed paths must come before parameterized ones.

NOTES

- Methods: `get` `post` `put` `patch` `delete` `head` `options` `trace`
- `@app.api_route(path, methods=[...])` handles several methods
- Same path, different methods → separate functions
- Function name becomes the default docs summary

UNDER THE HOOD

- Each decorator creates a Starlette `APIRoute`
- Path is compiled to a regex once at startup
- First matching route wins; no specificity sorting

CRUD ON ONE RESOURCE

```python
from fastapi import FastAPI, status

app = FastAPI()

@app.get("/todos")                          # list
def list_todos(): ...

@app.get("/todos/stats")                    # fixed path FIRST
def todo_stats(): ...

@app.get("/todos/{todo_id}")                # then the param path
def get_todo(todo_id: int): ...

@app.post("/todos", status_code=status.HTTP_201_CREATED)
def create_todo(todo: TodoIn): ...

@app.patch("/todos/{todo_id}")              # partial update
def update_todo(todo_id: int, todo: TodoPatch): ...

@app.delete("/todos/{todo_id}", status_code=204)
def delete_todo(todo_id: int) -> None: ...
```

CODE EXPLAINED

→ `/todos/stats` first — else `stats` hits the int route (422)

→ `status_code=` — success code for response and docs

→ `204` responses must not return a body

→ PUT replaces; PATCH changes only sent fields

OTHER FRAMEWORK

- Flask — `@app.get`, `@app.post`, or `@app.route(..., methods=[...])`
- Django — one view checks `request.method`; DRF ViewSets map methods

PREFER / AVOID

- Avoid verbs in paths like `/getTodos` · Prefer `GET /todos`
- Avoid one handler branching on method · Prefer one function per method

METHODS & DEFAULT CODES

METHOD	USE	TYPICAL CODE
GET	read	200
POST	create	201
PUT	replace	200
PATCH	partial update	200
DELETE	remove	204

DECORATOR ARGUMENTS

ARG	DOES
status_code	success status
response_model	output schema (15)
tags, summary	docs grouping/title
dependencies	run deps, ignore result
operation_id	stable name for client gen
responses	extra documented codes

⚠ COMMON MISTAKES

- Ordering `/users/{id}` above `/users/me`: `me` hits the param route
- Returning data from a `204` route: clients and proxies may choke
- Two functions with the same name in one module: the second silently replaces the first

✓ TIPS & BEST PRACTICES

- Group related operations with an `APIRouter` (sheet 25)
- Set `operation_id` if you generate typed clients
- Use `status` constants for readability

QUICK SUMMARY

- `@app.<method>(path)` — one operation per function
- registration order — first match wins
- `status_code=` — success code
- PUT vs PATCH — replace vs partial

Sheet 06

Path Parameters

/items/{item_id} · type conversion · Enum · {file_path:path} · 422 errors

DESCRIPTION

Path parameters are the `{name}` parts of a URL. Declare a function argument with the same name and a type, and FastAPI converts the string from the URL to that type, returning a **422** error with details if it can't. Use an `Enum` to restrict a parameter to fixed values, and `:path` to capture slashes.

NOTES

- Path params are always required
- Types: `int`, `float`, `str`, `bool`, `UUID`, `date`, Enums
- `Enum` values appear as a dropdown in `/docs`
- Constraints like `ge=1` come from `Path()` (sheet 09)

TYPED PATH PARAMETERS

```python
from enum import Enum
from uuid import UUID
from fastapi import FastAPI

class Status(str, Enum):                # str mixin → JSON-friendly
    open = "open"
    done = "done"

app = FastAPI()

@app.get("/todos/{todo_id}")
def get_todo(todo_id: int): ...         # /todos/x → 422

@app.get("/users/{user_id}")
def get_user(user_id: UUID): ...        # must be a valid UUID

@app.get("/todos/status/{status}")
def by_status(status: Status):          # only "open" or "done"
    return {"status": status, "is_done": status is Status.done}

@app.get("/files/{file_path:path}")
def read_file(file_path: str): ...      # "a/b/c.txt"
```

CODE EXPLAINED

→ `{todo_id}` + `todo_id: int` — names must match exactly

→ Bad input returns 422, never reaches your function

→ `class Status(str, Enum)` — limits values, documents them

→ `status is Status.done` — compare Enum members, not strings

→ `{file_path:path}` — Starlette converter that allows `/`

OTHER FRAMEWORK

- Flask — `/todos/<int:todo_id>`, `<uuid:id>`, `<path:p>` converters
- Django — `path("todos/<int:pk>/", ...)`

PREFER / AVOID

- Avoid `todo_id: str` then `int(todo_id)` · Prefer `todo_id: int`
- Avoid magic strings for fixed choices · Prefer `str, Enum`

CONVERSIONS

URL VALUE	TYPE	RESULT
`/7`	int	7
`/7.5`	int	422 error
`/2.5`	float	2.5
`/true or /1`	bool	True
`/2026-11-13`	date	date(2026,11,13)

422 ERROR BODY

FIELD	EXAMPLE
`type`	"int_parsing"
`loc`	["path", "todo_id"]
`msg`	"Input should be a valid integer..."
`input`	"abc"

⚠ COMMON MISTAKES

- Function arg named `id` but path says `{todo_id}`: it becomes a query param instead
- Using a plain `Enum` without `str`: serialization surprises in some tools
- Expecting `/files/{name}` to match `a/b.txt`: needs `:path`

✓ TIPS & BEST PRACTICES

- Prefer UUIDs in public URLs to avoid guessable IDs
- Add `Path(ge=1)` to reject zero and negatives (sheet 09)
- Test the 422 shape once; clients will depend on it

QUICK SUMMARY

- `{name}` + typed arg — path parameter
- conversion failure — automatic 422
- `str, Enum` — fixed set of values
- `{p:path}` — capture slashes

Query Parameters

non-path args · defaults · str | None = None · bool parsing · list[str] · Query()

DESCRIPTION

Any simple-typed function argument that is **not** in the path is read from the **query string**. Give it a default to make it optional; leave the default off to make it required. FastAPI parses `bool` leniently (`1`, `true`, `on`, `yes`), and needs `Query()` to accept a list of repeated values.

NOTES

- `skip: int = 0` → optional with default
- `q: str | None = None` → optional, may be missing
- `q: str` with no default → required (422 if missing)
- Order of args does not matter to FastAPI

FILTERING & PAGING

```python
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/todos")
def list_todos(
    owner: str,                          # required
    skip: int = 0,                       # optional, default 0
    limit: int = 20,
    done: bool | None = None,            # optional tri-state
    q: str | None = None,
    tag: Annotated[list[str] | None, Query()] = None,
):
    return {"owner": owner, "skip": skip, "done": done, "tag": tag}

# /todos?owner=bob                       → done None, tag None
# /todos?owner=bob&done=yes&skip=40      → done True, skip 40
# /todos?owner=bob&tag=home&tag=urgent   → tag ["home", "urgent"]
# /todos                                 → 422 owner missing
```

CODE EXPLAINED

- → `owner: str` — no default, so required
- → `bool | None = None` — lets you tell "not sent" from `False`
- → `list[str]` needs `Query()`; otherwise FastAPI expects a body
- → `Annotated[..., Query()]` — metadata without changing the type
- → Unknown query params are ignored by default

OTHER FRAMEWORK

- Flask — `request.args.get("skip", 0, type=int)`, `getlist("tag")`
- Django — `request.GET.get("skip")`, `getlist("tag")`

PREFER / AVOID

- Avoid `Optional[str]` in new code · Prefer `str | None`
- Avoid unbounded `limit` · Prefer `Query(le=100)` (sheet 09)

BOOL PARSING

SENT	VALUE
`true, 1, on, yes`	True
`false, 0, off, no`	False
`(missing)`	default
`maybe`	422 error

REQUIRED VS OPTIONAL

SIGNATURE	BEHAVIOR
`q: str`	required
`q: str = "x"`	optional, default "x"
`q: str | None = None`	optional, None if missing
`q: Annotated[str, Query()]`	required with metadata
`q: list[int] = Query([])`	repeatable, default empty

⚠ COMMON MISTAKES

- `tags: list[str] = []` without `Query()`: FastAPI treats it as a request body
- Using `if done:` when `done` may be `None`; use `if done is not None`
- Mutable default lists in plain functions; prefer `None` + `Query()`

✓ TIPS & BEST PRACTICES

- Group many filters into a Pydantic model with `Annotated[Filter, Query()]` (sheet 09)
- Use `skip` / `limit` or cursor params consistently across endpoints
- Document defaults in the signature; `/docs` shows them

QUICK SUMMARY

- non-path simple arg — query parameter
- default present — optional
- `X | None = None` — may be absent
- `list[T]` + `Query()` — repeated values

Request Body

Pydantic model param · JSON body · body + path + query · Body(embed=True) · multiple bodies

DESCRIPTION

Declare a parameter typed as a **Pydantic model** and FastAPI reads the **JSON request body**, validates it, and hands you a model instance. Path, query and body parameters can be mixed freely: FastAPI decides where each comes from by its name and type. Several model params become keys in one JSON object.

NOTES

- Model type → body; path name → path; simple type → query
- Wrong JSON or missing fields → 422 with field locations
- GET with a body is allowed but poorly supported by clients
- `Body()` marks a simple value as part of the body

UNDER THE HOOD

- Body bytes are read once and parsed with Pydantic's Rust JSON parser
- Starlette's `Request` caches the body for later reads
- Validation errors are collected, not raised one by one

MODEL IN, MODEL OUT

```python
from typing import Annotated
from pydantic import BaseModel
from fastapi import FastAPI, Body

class TodoIn(BaseModel):
    title: str
    notes: str | None = None
    priority: int = 3

app = FastAPI()

@app.post("/lists/{list_id}/todos")
def add_todo(list_id: int, todo: TodoIn, notify: bool = False):
    return {"list": list_id, **todo.model_dump(), "notify": notify}

@app.put("/todos/{todo_id}")
def update(todo_id: int, todo: TodoIn, user: User,
           importance: Annotated[int, Body()]): ...
# body: {"todo": {...}, "user": {...}, "importance": 5}

@app.post("/todos/embedded")
def embedded(todo: Annotated[TodoIn, Body(embed=True)]): ...
# body: {"todo": {"title": "Buy milk"}}
```

CODE EXPLAINED

- → `todo: TodoIn` — parsed from JSON body
- → `list_id` from the path, `notify` from the query string
- → `todo.model_dump()` — model → dict
- → Two models → each keyed by its param name
- → `Body(embed=True)` — wrap a single model under its name

OTHER FRAMEWORK

- Flask — `data = request.get_json()`, validate yourself
- Django — `json.loads(request.body)` or DRF serializers

PREFER / AVOID

- Avoid `request: Request` + `await request.json()` · Prefer a typed model param
- Avoid `dict` as the body type · Prefer a model so it is validated and documented

WHERE A PARAM COMES FROM

DECLARED AS	SOURCE
`name in path`	path
`int, str, bool...`	query
`BaseModel subclass`	body (JSON)
`Annotated[x, Body()]`	body key
`Annotated[x, Header()]`	header (10)

BODY SHAPES

SIGNATURE	EXPECTED JSON
`todo: TodoIn`	{"title": ...}
`todo: TodoIn, user: User`	{"todo":..., "user":...}
`Body(embed=True)`	{"todo": {...}}
`items: list[TodoIn]`	[{...}, {...}]

⚠ COMMON MISTAKES

- Sending form data to a JSON model endpoint: 422; forms need `Form()` (sheet 10)
- Adding a second model and breaking clients that sent the flat shape
- Reading `await request.json()` after FastAPI already parsed it, then mutating the wrong copy

✓ TIPS & BEST PRACTICES

- Use separate `In` and `Out` models (sheet 15)
- Send `Content-Type: application/json` from clients
- Use `list[Model]` for bulk create endpoints

QUICK SUMMARY

- model param — JSON body
- mix path, query and body in one signature
- two models — keyed JSON object
- `Body(embed=True)` — force the wrapper key

Sheet 09

Validation with Query, Path & Annotated

Annotated[int, Path(ge=1)] · Query(max_length=50) · pattern= · alias= · deprecated=True

DESCRIPTION

`Query()`, `Path()`, `Header()` and friends add **validation and docs metadata** to a parameter. Put them inside `Annotated[...]` so the real type stays visible and defaults stay normal Python defaults. Numeric bounds, string lengths, regex patterns, aliases and deprecation all live here.

NOTES

- `Annotated` style is the recommended one
- Old style `q: str = Query(None)` still works
- A Pydantic model with `Annotated[M, Query()]` groups query params
- Errors still come back as 422 with the failed rule

CONSTRAINED PARAMETERS

```python
from typing import Annotated
from pydantic import BaseModel, Field
from fastapi import FastAPI, Path, Query

app = FastAPI()
TodoId = Annotated[int, Path(ge=1, title="Todo ID")]   # reusable

@app.get("/todos/{todo_id}")
def get_todo(todo_id: TodoId): ...

@app.get("/search")
def search(
    q: Annotated[str, Query(min_length=2, max_length=50)],
    sort: Annotated[str, Query(pattern="^(asc|desc)$")] = "asc",
    page_size: Annotated[int, Query(alias="page-size", le=100)] = 20,
    old: Annotated[str | None, Query(deprecated=True)] = None,
): ...

class Filters(BaseModel):
    model_config = {"extra": "forbid"}   # reject unknown params
    limit: int = Field(20, gt=0, le=100)
    tags: list[str] = []

@app.get("/todos")
def list_todos(f: Annotated[Filters, Query()]): ...
```

CODE EXPLAINED

- → `Path(ge=1)` — 0 or negatives → 422
- → `TodoId` alias — reuse the same rules on many routes
- → `pattern=` — regex the value must match
- → `alias="page-size"` — URL name differs from Python name
- → Query model — one object for many filters

OTHER FRAMEWORK

- Flask — validate manually or with Marshmallow / Pydantic
- Django — forms or DRF serializers validate query data

PREFER / AVOID

- Avoid `q: str = Query(None)` · Prefer `Annotated[str | None, Query()] = None`
- Avoid repeating the same `Path(ge=1)` · Prefer a type alias

NUMBER RULES

ARG	MEANING
`gt / ge`	> / >=
`lt / le`	< / <=
`multiple_of`	divisible by
`allow_inf_nan`	float infinity/NaN

STRING & DOCS RULES

ARG	MEANING
`min_length / max_length`	length bounds
`pattern`	regex
`alias`	external name
`title, description`	docs text
`examples`	docs examples
`deprecated`	mark in docs

⚠ COMMON MISTAKES

- Using `regex=`: renamed to `pattern=` in current versions
- Putting a default inside `Query(...)` and also after `=`: pick one (after `=` with `Annotated`)
- Forgetting `alias` makes Python-style names show in URLs

✓ TIPS & BEST PRACTICES

- Define `Annotated` aliases in `deps.py` and reuse them
- Use `extra="forbid"` on query models to catch typos
- Keep max page sizes server-side, not just in the client

QUICK SUMMARY

- `Annotated[T, Query(...)]` — validated query param
- `Path(ge=1)` — bounded path param
- `pattern`, `min_length` — string rules
- query model — grouped filters

Headers, Cookies, Forms & Files

DESCRIPTION

Not everything arrives as JSON. `Header()` and `Cookie()` read those parts of the request; `Form()` reads HTML form fields; `File()` and **`UploadFile`** handle multipart uploads. Forms and files need the **`python-multipart`** package (included in `fastapi[standard]`). A request can carry form fields **or** JSON, not both.

NOTES

- `user_agent` header param matches `User-Agent` (underscores → hyphens)
- `UploadFile` spools to disk past ~1 MB; `bytes` loads into memory
- Multiple files: `list[UploadFile]`
- Files + JSON together: send JSON as a form field

NON-JSON INPUTS

```python
from typing import Annotated
from fastapi import FastAPI, Header, Cookie, Form, File, UploadFile

app = FastAPI()

@app.get("/whoami")
def whoami(
    user_agent: Annotated[str | None, Header()] = None,
    x_request_id: Annotated[str | None, Header()] = None,
    session: Annotated[str | None, Cookie()] = None,
): ...

@app.post("/login")
def login(username: Annotated[str, Form()],
          password: Annotated[str, Form()]): ...

@app.post("/avatars")
async def upload(file: UploadFile, caption: Annotated[str, Form()] = ""):
    data = await file.read()                # bytes
    return {"name": file.filename, "type": file.content_type,
            "size": len(data)}

@app.post("/attachments")
async def many(files: list[UploadFile]): ...
```

CODE EXPLAINED

- → `Header()` — `x_request_id` reads `X-Request-Id`
- → `Cookie()` — reads a cookie by name
- → `Form()` — `application/x-www-form-urlencoded` or multipart field
- → `UploadFile` — `.filename`, `.content_type`, `await .read()`
- → `list[UploadFile]` — several files under one field

OTHER FRAMEWORK

- Flask — `request.headers`, `request.cookies`, `request.form`, `request.files`
- Django — `request.headers`, `request.COOKIES`, `request.POST`, `request.FILES`

PREFER / AVOID

- Avoid `file: bytes` for large uploads · Prefer `UploadFile`
- Avoid trusting `file.filename` as a path · Prefer generating your own name

UPLOADFILE API

MEMBER	DOES
`filename`	client-sent name
`content_type`	e.g. image/png
`size`	bytes, if known
`await read(n)`	read bytes
`await seek(0)`	rewind
`file`	underlying SpooledTemporaryFile

PARAM FUNCTIONS

FUNCTION	READS
`Header()`	request headers
`Cookie()`	cookies
`Form()`	form fields
`File()`	file as bytes
`UploadFile`	file object

⚠ COMMON MISTAKES

- Mixing a JSON model with `Form()` params: the body can only be one encoding
- Missing `python-multipart`: FastAPI raises at startup for form routes
- Saving uploads with the client filename: path traversal risk

✓ TIPS & BEST PRACTICES

- Check `content_type` and size before saving uploads
- Use `convert_underscores=False` on `Header()` for odd header names
- Stream large files to disk with `shutil.copyfileobj(file.file, out)`

QUICK SUMMARY

- `Header()` / `Cookie()` — request metadata
- `Form()` — form fields
- `UploadFile` — streamed file uploads
- form vs JSON — one encoding per request

Pydantic BaseModel

class Item(BaseModel) · type coercion · defaults · model_dump() · model_validate() · model_dump_json()

DESCRIPTION

A **Pydantic model** is a class that declares fields with type hints. Creating one validates and **coerces** the input ("3" becomes 3 for an `int` field) or raises `ValidationError`. FastAPI uses models for request bodies, responses and settings. Pydantic **v2** is the current major version, with a Rust core and `model_*` method names.

NOTES

- Fields without defaults are required
- Strict mode turns off coercion: `ConfigDict(strict=True)`
- Models are mutable unless `frozen=True`
- v1 names (`.dict()` , `.parse_obj()`) are deprecated

UNDER THE HOOD

- `pydantic-core` (Rust) builds a validator from your class at import time
- Validation runs in Rust, then fills the Python object
- `model_fields` holds the schema; FastAPI turns it into JSON Schema

CREATE, VALIDATE, DUMP

```python
from datetime import datetime
from pydantic import BaseModel, ValidationError

class Todo(BaseModel):
    id: int
    title: str
    done: bool = False
    created: datetime | None = None

t = Todo(id="7", title="Buy milk")  # "7" → 7
t.model_dump()     # {'id': 7, 'title': 'Buy milk', ...}
t.model_dump(exclude_none=True)    # drops created
t.model_dump_json()                # '{"id":7,...}'

Todo.model_validate({"id": 1, "title": "x"})
Todo.model_validate_json('{"id":1,"title":"x"}')
t2 = t.model_copy(update={"done": True})

try:
    Todo(id="abc")
except ValidationError as e:
    e.error_count()          # 2: bad id, missing title
```

CODE EXPLAINED

→ Class fields + hints — the schema

→ Constructor validates and coerces

→ `model_dump()` — to dict; `model_dump_json()` — to JSON string

→ `model_validate()` — from dict or object

→ `model_copy(update=)` — changed copy

OTHER FRAMEWORK

- Flask — no built-in models; pair with Pydantic or Marshmallow
- Django — `forms.Form` or DRF `Serializer` fill this role

PREFER / AVOID

- Avoid v1 `.dict()` / `.json()` · Prefer `model_dump()` / `model_dump_json()`
- Avoid raw dicts passed between layers · Prefer models with real types

V1 → V2 NAMES

V1	V2
`.dict()`	.model_dump()
`.json()`	.model_dump_json()
`parse_obj()`	model_validate()
`parse_raw()`	model_validate_json()
`.copy()`	.model_copy()
`class Config`	model_config = ConfigDict()

COERCION (LAX MODE)

INPUT	FIELD	RESULT
`"3"`	int	3
`3.0`	int	3
`3.5`	int	error
`"yes"`	bool	True
`"2026-11-13"`	date	date

⚠ COMMON MISTAKES

- Using v1 tutorials: `orm_mode` , `.dict()` and `@validator` changed in v2
- Assuming assignment re-validates: it does not unless `validate_assignment=True`
- Mutable defaults like `tags: list = []` are fine in Pydantic, but not in dataclasses

✓ TIPS & BEST PRACTICES

- Use `model_dump(mode="json")` to get JSON-safe types (dates as strings)
- Turn on `strict=True` for money or ID fields
- Inspect `Model.model_json_schema()` to see what the docs will show

QUICK SUMMARY

- `BaseModel` — typed, validated data class
- constructor — validate + coerce
- `model_dump()` / `model_validate()` — in and out
- Pydantic v2 — `model_*` method names

Field & Constraints

DESCRIPTION

`Field()` adds **constraints, defaults and docs** to a model field, the same way `Query()` does for parameters. Pydantic also ships ready-made types such as **`EmailStr`**, **`HttpUrl`**, `PositiveInt` and `SecretStr`. Constraints can live in `Field()` or inside `Annotated[...]`, which lets you build reusable constrained types.

CONSTRAINED MODEL

```python
from datetime import datetime, timezone
from decimal import Decimal
from typing import Annotated
from uuid import UUID, uuid4
from pydantic import (BaseModel, Field, EmailStr,
                      HttpUrl, SecretStr)

Title = Annotated[str, Field(min_length=1, max_length=100)]

class SignUp(BaseModel):
    id: UUID = Field(default_factory=uuid4)
    email: EmailStr
    password: SecretStr = Field(min_length=12)
    name: Title
    age: int = Field(ge=13, le=120)
    website: HttpUrl | None = None
    balance: Decimal = Field(0, max_digits=10, decimal_places=2)
    tags: list[str] = Field(default_factory=list, max_length=5)
    joined: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc))

    model_config = {"json_schema_extra": {"examples": [{
        "email": "bob@example.com", "password": "horse-battery",
        "name": "Bob", "age": 40}]}}
```

NOTES

- `EmailStr` needs `email-validator` (in `fastapi[standard]`)
- `default_factory` runs per instance (lists, UUIDs, timestamps)
- `examples=[...]` pre-fills "Try it out" in `/docs`
- `SecretStr` hides the value in reprs and logs

CODE EXPLAINED

- → `Title` — reusable constrained string
- → `default_factory=uuid4` — new ID per instance
- → `SecretStr` — use `.get_secret_value()` to read
- → `max_length` on a list — max items
- → `json_schema_extra` examples — shown in `/docs`

OTHER FRAMEWORK

- Flask — WTForms validators: `Length`, `Email`, `NumberRange` (sheet 50)
- Django — `max_length`, `validators=[...]` on model/form fields

PREFER / AVOID

- Avoid `default=datetime.now()` (evaluated once) · Prefer `default_factory`
- Avoid `float` for money · Prefer `Decimal` with `decimal_places`

FIELD() ARGUMENTS

ARG	PURPOSE
`default / default_factory`	default value
`gt, ge, lt, le`	number bounds
`min_length, max_length`	str/list size
`pattern`	regex
`alias`	external key name
`description, examples`	docs

BUILT-IN TYPES

TYPE	ACCEPTS
`EmailStr`	valid email
`HttpUrl / AnyUrl`	URLs
`SecretStr`	hidden string
`PositiveInt`	> 0
`UUID4`	version-4 UUID
`IPvAnyAddress`	IPv4/IPv6

⚠ COMMON MISTAKES

- `created: datetime = datetime.now()` : every instance shares the import-time value
- Logging a `SecretStr` with `.get_secret_value()` defeats the point
- Forgetting `email-validator` when installing plain `fastapi`

✓ TIPS & BEST PRACTICES

- Put shared `Annotated` types in one module (`types.py`)
- Use `alias` + `populate_by_name` for camelCase APIs (sheet 16)
- Keep examples realistic; they double as documentation

QUICK SUMMARY

- `Field()` — constraints, defaults, docs
- `default_factory` — per-instance defaults
- `EmailStr`, `HttpUrl`, `SecretStr` — ready-made types
- `Annotated` — reusable constrained types

Custom Validators

DESCRIPTION

When `Field()` rules aren't enough, write **validators**. A `@field_validator` checks or transforms one field; a `@model_validator` sees the whole model, which is how you compare two fields. Raise `ValueError` with a clear message and FastAPI turns it into a 422 entry. `@computed_field` adds read-only values to the output.

NOTES

- Validators are `@classmethod` s in v2
- `mode="before"` sees raw input; `"after"` sees typed values
- Return the (possibly changed) value from a field validator
- Return `self` from an after model validator

FIELD, MODEL & COMPUTED

```python
from datetime import date
from typing import Annotated
from pydantic import (BaseModel, AfterValidator, computed_field,
                      field_validator, model_validator)

def no_spaces(v: str) -> str:
    if " " in v: raise ValueError("no spaces allowed")
    return v

class Trip(BaseModel):
    code: Annotated[str, AfterValidator(no_spaces)]
    city: str
    start: date; end: date

    @field_validator("city")
    @classmethod
    def tidy_city(cls, v: str) -> str:
        return v.strip().title()      # " ames" → "Ames"

    @model_validator(mode="after")
    def check_dates(self):
        if self.end < self.start: raise ValueError("end < start")
        return self

    @computed_field
    @property
    def nights(self) -> int:
        return (self.end - self.start).days
```

CODE EXPLAINED

- → `AfterValidator(fn)` — reusable validator inside `Annotated`
- → `@field_validator("city")` — runs for that field
- → `@model_validator(mode="after")` — cross-field checks
- → `ValueError` → 422 item with your message
- → `@computed_field` — included in `model_dump()` and docs

OTHER FRAMEWORK

- Flask — WTForms `validate_<field>()` methods (sheet 50)
- Django — `clean_<field>()` and `clean()` on forms

PREFER / AVOID

- Avoid v1 `@validator` / `@root_validator` · Prefer `@field_validator` / `@model_validator`
- Avoid raising `HTTPException` in validators · Prefer `ValueError`

VALIDATOR MODES

MODE	GETS	USE FOR
`before`	raw input	normalize strings, parse odd formats
`after`	typed value	business rules
`wrap`	value + handler	skip or retry validation
`plain`	raw input	replace validation entirely

ANNOTATED VALIDATORS

NAME	RUNS
`BeforeValidator(fn)`	before type check
`AfterValidator(fn)`	after type check
`PlainValidator(fn)`	instead of type check
`WrapValidator(fn)`	around type check

⚠ COMMON MISTAKES

- Forgetting `return v` : the field becomes `None`
- Missing `@classmethod` under `@field_validator`
- Doing DB lookups in validators: slow, and validators can't use dependencies

✓ TIPS & BEST PRACTICES

- Check uniqueness (email taken) in the route or a dependency, not the model
- Use `mode="before"` to accept `"a,b,c"` and split into a list
- Keep messages user-facing; they go straight to API clients

QUICK SUMMARY

- `@field_validator` — one field
- `@model_validator` — whole model, cross-field
- `AfterValidator` — reusable via `Annotated`
- `@computed_field` — derived output value

Sheet 14

Nested Models & Special Types

nested models · list[Item] · dict[str, int] · Literal · datetime / UUID / Decimal · discriminated unions

DESCRIPTION

Fields can be **other models**, lists and dicts of models, or special types. Pydantic validates the whole tree and reports errors with a full location path like `items.2.price`. `Literal` restricts a field to exact values, and a **discriminated union** picks the right model from a tag field such as `type`.

NOTES

- Nested errors show the path: `["body", "items", 2, "price"]`
- `dict[str, int]` validates keys and values
- `Literal["a", "b"]` is a lightweight Enum
- `Field(discriminator="type")` makes unions fast and clear

AN ORDER WITH LINE ITEMS

```python
from datetime import datetime
from decimal import Decimal
from typing import Annotated, Literal
from pydantic import BaseModel, Field

class Address(BaseModel):
    city: str
    zip: str = Field(pattern=r"^\d{5}$")

class LineItem(BaseModel):
    sku: str
    qty: int = Field(gt=0)
    price: Decimal

class Card(BaseModel):   type: Literal["card"];   last4: str
class PayPal(BaseModel): type: Literal["paypal"]; email: str

class Order(BaseModel):
    ship_to: Address
    items: list[LineItem] = Field(min_length=1)
    payment: Annotated[Card | PayPal, Field(discriminator="type")]
    meta: dict[str, str] = {}
    placed: datetime
```

CODE EXPLAINED

- → `ship_to: Address` — nested object
- → `list[LineItem]` + `min_length=1` — at least one item
- → `Literal["card"]` — exact value only
- → `discriminator="type"` — choose model by `type`
- → `datetime` — accepts ISO 8601 strings

OTHER FRAMEWORK

- Flask — nested JSON is plain dicts unless you add a schema library
- Django — DRF nested serializers (`many=True`)

PREFER / AVOID

- Avoid `dict` for structured data · Prefer a nested model
- Avoid untagged `A | B` unions · Prefer a discriminator field

SPECIAL TYPES

TYPE	ACCEPTS / GIVES
`datetime`	ISO string or epoch
`date / time`	ISO strings
`timedelta`	seconds or ISO 8601
`UUID`	UUID string
`Decimal`	exact numbers
`bytes`	str or bytes

COLLECTION TYPES

HINT	JSON
`list[int]`	[1, 2]
`set[str]`	unique list
`tuple[int, str]`	[1, "a"]
`dict[str, float]`	{"a": 1.5}
`Literal["x", "y"]`	"x"

⚠ COMMON MISTAKES

- Untagged unions match the first model that fits: wrong type silently chosen
- `float` prices: `0.1 + 0.2` problems; use `Decimal`
- Naive `datetime` values: store and return timezone-aware UTC

✓ TIPS & BEST PRACTICES

- Reuse nested models across In/Out schemas
- Let the error `loc` path drive form-field highlighting on the client
- Use `set[str]` for tags to dedupe automatically

QUICK SUMMARY

- nested model — field typed as another model
- `list[Model]` — arrays of objects
- `Literal` — exact allowed values
- discriminated union — tag picks the model

Response Models & Filtering

DESCRIPTION

A **response model** controls what leaves your API. FastAPI validates the return value against it and **filters out** fields not in the model, which is how you keep password hashes and internal columns private. Declare it with a return annotation (`-> UserOut`) or `response_model=` when the function returns something else, like an ORM object.

NOTES

- Return type annotation is the preferred style
- `response_model=` wins if both are set
- `from_attributes=True` lets models read ORM objects
- `response_model=None` disables validation for that route

UNDER THE HOOD

- FastAPI serializes the return value through the response model's Pydantic serializer
- Extra attributes are dropped during that serialization
- Then `jsonable_encoder` rules make it JSON-safe

IN / DB / OUT MODELS

```python
from pydantic import BaseModel, ConfigDict, EmailStr

class UserBase(BaseModel):
    email: EmailStr
    name: str

class UserIn(UserBase):
    password: str                    # in only, never returned

class UserOut(UserBase):
    model_config = ConfigDict(from_attributes=True)
    id: int

@app.post("/users", status_code=201)
def create_user(user: UserIn) -> UserOut:
    db_user = save(user)             # ORM obj w/ hashed_password
    return db_user                   # → id, email, name only

@app.get("/users", response_model=list[UserOut])
def list_users():
    return session.scalars(select(User)).all()

@app.patch("/users/{id}", response_model_exclude_unset=True)
def patch_user(id: int, data: UserPatch) -> UserOut: ...
```

CODE EXPLAINED

- → `UserIn` / `UserOut` — inherit shared fields from `UserBase`
- → `-> UserOut` — documents and filters the output
- → `from_attributes=True` — read `.email` from ORM objects
- → `list[UserOut]` — arrays work the same way

OTHER FRAMEWORK

- Flask — build the dict yourself or use `marshmallow` `dump()`
- Django — DRF serializer `fields = [...]` controls output

PREFER / AVOID

- Avoid returning DB models directly without a schema · Prefer an `Out` model
- Avoid deleting keys from dicts to hide data · Prefer a model that omits them

RESPONSE MODEL OPTIONS

OPTION	EFFECT
`response_model_exclude_unset`	only fields actually set
`response_model_exclude_none`	drop None values
`response_model_exclude={...}`	drop named fields
`response_model_include={...}`	keep only named fields
`response_model_by_alias`	use aliases (default True)

WHICH TO USE

SITUATION	USE
`returns the model`	-> Model
`returns ORM object`	response_model= + from_attributes
`returns Response`	response_model=None
`list result`	-> list[Model]

⚠ COMMON MISTAKES

- Returning a SQLAlchemy object without `from_attributes=True`: validation error
- Annotating `-> dict` and losing filtering and docs
- Using `include` / `exclude` everywhere instead of separate models

✓ TIPS & BEST PRACTICES

- Name schemas by role: `Create`, `Update`, `Read`
- Make all `Update` fields optional and use `exclude_unset`
- Check `/docs` → Schemas to confirm no secret fields leak

QUICK SUMMARY

- `-> Out` — validate + filter output
- In / Out models — never return passwords
- `from_attributes` — accept ORM objects
- `exclude_unset` — partial responses

model_config & Settings

DESCRIPTION

`model_config = ConfigDict(...)` changes how a model behaves: reject unknown keys, use camelCase aliases, freeze instances. **`pydantic-settings`** applies the same idea to configuration: a `BaseSettings` class reads **environment variables** and `.env` files, validates them, and fails fast at startup if something is missing.

NOTES

- `pip install pydantic-settings` (separate package)
- Env var names are case-insensitive by default
- Real env vars override `.env` file values
- Never commit `.env`; commit `.env.example`

CONFIG + SETTINGS

```python
from functools import lru_cache
from typing import Annotated
from fastapi import Depends
from pydantic import BaseModel, ConfigDict
from pydantic.alias_generators import to_camel
from pydantic_settings import BaseSettings, SettingsConfigDict

class ApiModel(BaseModel):
    model_config = ConfigDict(alias_generator=to_camel,
                              populate_by_name=True, extra="forbid")
    due_date: str                    # JSON key "dueDate"

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="APP_")
    database_url: str                # APP_DATABASE_URL (required)
    debug: bool = False
    secret_key: str

@lru_cache
def get_settings() -> Settings:
    return Settings()

SettingsDep = Annotated[Settings, Depends(get_settings)]
```

CODE EXPLAINED

- → `alias_generator=to_camel` — snake in Python, camel in JSON
- → `populate_by_name` — accept either name on input
- → `extra="forbid"` — unknown keys → 422
- → `env_prefix="APP_"` — read `APP_DEBUG`, etc.
- → `@lru_cache` — build settings once, reuse

OTHER FRAMEWORK

- Flask — `app.config.from_prefixed_env()` (sheet 52)
- Django — `settings.py` + `os.environ` or `django-environ`

PREFER / AVOID

- Avoid `os.getenv()` scattered in code · Prefer one `Settings` class
- Avoid a global `settings = Settings()` in tests · Prefer `get_settings` + override

CONFIGDICT OPTIONS

OPTION	EFFECT
`extra="forbid"/"ignore"/"allow"`	unknown keys
`frozen=True`	immutable, hashable
`strict=True`	no coercion
`validate_assignment`	check on set
`str_strip_whitespace`	trim strings
`from_attributes`	read objects

SETTINGSCONFIGDICT

OPTION	EFFECT
`env_file`	load .env
`env_prefix`	name prefix
`env_nested_delimiter="__"`	DB__HOST → db.host
`case_sensitive`	exact env names
`secrets_dir`	Docker secrets files

⚠ COMMON MISTAKES

- Calling `Settings()` in many places: rereads env each time
- Committing `.env` with real secrets
- `extra="forbid"` on settings, then crashing on unrelated env vars from `.env`

✓ TIPS & BEST PRACTICES

- Override `get_settings` in tests with `app.dependency_overrides`
- Use `SecretStr` for keys so they don't print
- Validate settings at startup in the lifespan (sheet 26)

QUICK SUMMARY

- `ConfigDict` — model behavior switches
- aliases — camelCase JSON, snake_case Python
- `BaseSettings` — typed env config
- `@lru_cache` — one settings instance

Response Types

JSONResponse · HTMLResponse · PlainTextResponse · RedirectResponse · FileResponse · StreamingResponse

DESCRIPTION

By default FastAPI wraps your return value in a **JSONResponse**. Return a Response subclass yourself to send HTML, plain text, redirects, files or streams. Returning a Response **skips** response-model validation, so you control the exact bytes. Set response_class= on the decorator to change the default and the docs.

NOTES

- All live in `fastapi.responses` (re-exported from Starlette)
- `FileResponse` streams from disk and sets `Content-Length`
- `RedirectResponse` defaults to 307 (keeps method)
- `ORJSONResponse` is faster JSON (needs `orjson`)

CHOOSING A RESPONSE

```python
from fastapi.responses import (HTMLResponse, PlainTextResponse,
    RedirectResponse, FileResponse, StreamingResponse, JSONResponse)

@app.get("/hello", response_class=HTMLResponse)
def hello():
    return "<h1>Hello</h1>"                  # str sent as HTML

@app.get("/old")
def old():
    return RedirectResponse("/new", status_code=301)

@app.get("/report.pdf")
def report():
    return FileResponse("files/report.pdf", filename="report.pdf")

@app.get("/teapot")
def teapot():
    return JSONResponse({"error": "teapot"}, status_code=418,
                        headers={"X-Brew": "no"})
```

CODE EXPLAINED

- → `response_class=` — sets the default class and docs media type
- → `RedirectResponse(url, 301)` — permanent redirect
- → `FileResponse(filename=)` — download with a name
- → `JSONResponse(..., status_code, headers)` — full control

OTHER FRAMEWORK

- Flask — return a str, `redirect()`, `send_file()`, `Response(...)`
- Django — `HttpResponse`, `JsonResponse`, `FileResponse`, `redirect()`

PREFER / AVOID

- Avoid `JSONResponse(content=model)` · Prefer returning the model (validated, filtered)
- Avoid reading a whole file into memory · Prefer `FileResponse`

RESPONSE CLASSES

CLASS	MEDIA TYPE
JSONResponse	application/json
HTMLResponse	text/html
PlainTextResponse	text/plain
RedirectResponse	Location header
FileResponse	guessed from file
StreamingResponse	any, chunked (39)

REDIRECT CODES

CODE	MEANING
301	moved permanently
302	found (may change to GET)
303	see other: POST → GET
307	temporary, keep method
308	permanent, keep method

⚠ COMMON MISTAKES

- Returning a `JSONResponse` with a datetime: not JSON-serializable without `jsonable_encoder`
- Using 302 after a form POST: use 303 so the browser does a GET
- Expecting response models to filter a returned `JSONResponse`

✓ TIPS & BEST PRACTICES

- Use `response_class=ORJSONResponse` app-wide for speed (sheet 44)
- Pass `media_type=` to `Response` for custom types like CSV
- Use `FileResponse` with `headers={"Cache-Control": ...}` for static downloads

QUICK SUMMARY

- default — `JSONResponse`
- return a `Response` — exact control, no filtering
- `response_class=` — change default + docs
- `FileResponse` / `RedirectResponse` — common cases

Status Codes, Headers & Cookies

DESCRIPTION

To change the status code, headers or cookies **while still returning a model**, declare a parameter of type `Response`. FastAPI injects a temporary response; anything you set on it is merged into the final one. Use `status_code=` on the decorator for the normal case and `responses=` to document other codes.

NOTES

- `from fastapi import status` has named constants
- Setting `response.status_code` overrides the decorator for that call
- `set_cookie` and `delete_cookie` take standard cookie options
- `responses=` only documents; it doesn't change behavior

TWEAK THE RESPONSE

```python
from fastapi import Response, status

@app.put("/todos/{todo_id}", status_code=200,
         responses={201: {"description": "Created"},
                    404: {"description": "Not found"}})
def upsert(todo_id: int, todo: TodoIn, response: Response) -> TodoOut:
    if not exists(todo_id):
        response.status_code = status.HTTP_201_CREATED
    response.headers["X-Todo-Version"] = "3"
    return save(todo_id, todo)

@app.post("/login")
def login(form: LoginForm, response: Response):
    token = make_session(form)
    response.set_cookie("session", token, httponly=True,
                        secure=True, samesite="lax",
                        max_age=60 * 60 * 24 * 7)
    return {"ok": True}

@app.post("/logout")
def logout(response: Response):
    response.delete_cookie("session")
```

CODE EXPLAINED

- → `response: Response` — injected, merged into the result
- → `response.status_code` — per-call override
- → `response.headers[...]` — add custom headers
- → `set_cookie(httponly, secure, samesite)` — safe session cookie
- → `responses={...}` — extra codes in `/docs`

OTHER FRAMEWORK

- Flask — return `(body, 201, {"X-H": "v"})` or `make_response()`
- Django — `resp = JsonResponse(...); resp.status_code = 201`

PREFER / AVOID

- Avoid building a `JSONResponse` just to add a header · Prefer the `Response` param
- Avoid cookies without `httponly` / `secure` · Prefer safe defaults

COMMON STATUS CONSTANTS

CONSTANT	CODE
`HTTP_200_OK`	200
`HTTP_201_CREATED`	201
`HTTP_204_NO_CONTENT`	204
`HTTP_400_BAD_REQUEST`	400
`HTTP_401_UNAUTHORIZED`	401
`HTTP_404_NOT_FOUND`	404

SET_COOKIE OPTIONS

OPTION	PURPOSE
`max_age / expires`	lifetime
`httponly=True`	no JS access
`secure=True`	HTTPS only
`samesite="lax"`	CSRF protection
`domain, path`	scope

⚠ COMMON MISTAKES

- Returning a new `Response` and also setting the injected one: injected changes are lost
- `samesite="none"` without `secure=True`: browsers reject it
- Using 200 for creates: clients expect 201 + the new resource

✓ TIPS & BEST PRACTICES

- Add a `Location` header on 201 responses
- Document every non-200 code clients must handle
- Prefer `status` constants over bare numbers in reviews

QUICK SUMMARY

- `status_code=` — default success code
- `response: Response` — set code, headers, cookies
- `set_cookie()` — httponly + secure + samesite
- `responses=` — document other codes

HTTPException & Exception Handlers

raise HTTPException(404) · detail= · headers= · @app.exception_handler · custom exceptions

DESCRIPTION

Raise **`HTTPException`** to stop a request and send an error response: FastAPI returns `{"detail": ...}` with your status code. For domain errors, define your own exception classes and register an **exception handler** that turns them into responses. That keeps business code free of HTTP details.

NOTES

- `raise`, don't `return`, an `HTTPException`
- `detail` can be any JSON-able value (str, dict, list)
- Uncaught exceptions → 500 and a server log entry
- Use `fastapi.HTTPException` (allows `headers=`)

ERRORS TWO WAYS

```python
from fastapi import HTTPException, Request, status
from fastapi.responses import JSONResponse

@app.get("/todos/{todo_id}")
def get_todo(todo_id: int) -> TodoOut:
    todo = repo.get(todo_id)
    if todo is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND,
                            detail=f"Todo {todo_id} not found")
    return todo

class QuotaExceeded(Exception):          # domain error
    def __init__(self, limit: int):
        self.limit = limit

@app.exception_handler(QuotaExceeded)
async def quota_handler(request: Request, exc: QuotaExceeded):
    return JSONResponse(status_code=429,
        content={"detail": f"Limit of {exc.limit} reached"},
        headers={"Retry-After": "3600"})

# anywhere in services: raise QuotaExceeded(limit=100)
```

CODE EXPLAINED

- → `raise HTTPException(404, detail=)` — immediate error response
- → Response body: `{"detail": "Todo 7 not found"}`
- → `QuotaExceeded` — plain Python exception, no HTTP coupling
- → `@app.exception_handler(Cls)` — maps it to a response
- → Handlers get the `Request` and the exception

OTHER FRAMEWORK

- Flask — `abort(404)` and `@app.errorhandler(404)` (sheet 53)
- Django — `raise Http404` or `get_object_or_404()`

PREFER / AVOID

- Avoid `return {"error": ...}` with 200 · Prefer raising with the right status
- Avoid `HTTPException` deep in services · Prefer domain exceptions + handlers

ERROR STATUS CODES

CODE	WHEN
400	bad request (semantic)
401	not authenticated
403	authenticated, not allowed
404	not found
409	conflict (duplicate)
429	rate limited

HANDLER TARGETS

REGISTER FOR	CATCHES
`YourException`	your domain error
`StarletteHTTPException`	all HTTP errors, incl. 404 routes
`RequestValidationError`	422s (sheet 20)
`Exception`	everything else → 500

⚠ COMMON MISTAKES

- `return HTTPException(...)`: returns 200 with a weird body
- Catching `Exception` in every route and hiding real bugs
- Leaking stack traces or SQL in `detail`

✓ TIPS & BEST PRACTICES

- Register a handler for `starlette.exceptions.HTTPException` to restyle all errors
- Use `headers={"WWW-Authenticate": "Bearer"}` on 401s
- Keep error bodies one consistent shape (sheet 20)

QUICK SUMMARY

- `raise HTTPException(code, detail)` — error response
- domain exception + handler — clean separation
- `@app.exception_handler` — custom mapping
- uncaught — 500, logged

Sheet 20

Validation Errors & Error Formats

422 shape · RequestValidationError · override handler · jsonable_encoder · error envelopes

DESCRIPTION

When input fails validation FastAPI raises **`RequestValidationError`** and returns **422** with a `detail` list: one entry per problem, each with `type`, `loc`, `msg` and `input`. You can override the handler to reshape errors into your API's standard **error envelope**, change the status to 400, or log bad requests.

NOTES

- `loc` starts with the source: `body`, `query`, `path`, `header`
- Pydantic errors in your own code are `ValidationError` (500 if uncaught)
- `exc.errors()` gives the list; `exc.body` the raw body
- `jsonable_encoder` makes errors JSON-safe

CUSTOM ERROR ENVELOPE

```python
from fastapi import Request
from fastapi.encoders import jsonable_encoder
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

def envelope(code: str, message: str, fields=None, status=400):
    return JSONResponse(status_code=status, content=jsonable_encoder(
        {"error": {"code": code, "message": message, "fields": fields}}))

@app.exception_handler(RequestValidationError)
async def on_invalid(request: Request, exc: RequestValidationError):
    fields = {".".join(map(str, e["loc"][1:])): e["msg"]
              for e in exc.errors()}
    return envelope("invalid_input", "Check the fields", fields, 422)

@app.exception_handler(StarletteHTTPException)
async def on_http(request: Request, exc: StarletteHTTPException):
    return envelope(f"http_{exc.status_code}", str(exc.detail),
                    status=exc.status_code)
```

CODE EXPLAINED

- → `exc.errors()` — list of error dicts
- → `loc[1:]` — drop `body` to get the field path
- → `jsonable_encoder` — converts dates, Decimals, sets
- → StarletteHTTPException handler — also catches unknown-route 404s
- → Every error now has one shape

OTHER FRAMEWORK

- Flask — return your own error JSON from `@app.errorhandler`
- Django — DRF `EXCEPTION_HANDLER` setting

PREFER / AVOID

- Avoid ad-hoc error shapes per route · Prefer one envelope everywhere
- Avoid echoing `input` for password fields · Prefer stripping sensitive inputs

DEFAULT 422 ENTRY

KEY	EXAMPLE
type	"missing"
loc	["body", "email"]
msg	"Field required"
input	{…}
ctx	{"ge": 1} (for limits)

COMMON ERROR TYPES

TYPE	CAUSE
missing	required field absent
int_parsing	not an integer
string_too_short	min_length
greater_than_equal	ge rule
value_error	your ValueError
extra_forbidden	unknown key

⚠ COMMON MISTAKES

- Overriding the 422 handler and dropping the field locations clients rely on
- Registering for FastAPI's `HTTPException` only: route 404s use Starlette's
- Logging full request bodies with passwords

✓ TIPS & BEST PRACTICES

- Version your error format with the API
- Use RFC 9457 Problem Details if you want a standard shape
- Reuse FastAPI's default handlers: `from fastapi.exception_handlers import ...`

QUICK SUMMARY

- 422 — `detail` list of `type/loc/msg`
- `RequestValidationError` — override to reshape
- Starlette `HTTPException` — catch all HTTP errors
- one envelope — predictable clients

Depends Basics

DESCRIPTION

Dependency injection lets a path operation declare "I need X" and have FastAPI build X for it. A dependency is any callable; its parameters are resolved exactly like a route's (path, query, body, other dependencies). Use it for shared query params, DB sessions, the current user, settings, and permission checks.

NOTES

- Dependencies can be `def` or `async def`
- Their params appear in `/docs` as if on the route
- Same dependency used twice in one request runs once
- Wrap in an `Annotated` alias to reuse with one name

UNDER THE HOOD

- FastAPI builds a dependency tree per route at startup
- Per request it solves the tree, caching results by callable
- `def` dependencies run in the threadpool, like `def` routes

SHARED PAGING DEPENDENCY

```python
from typing import Annotated
from fastapi import Depends, FastAPI, Query

app = FastAPI()

def paging(skip: Annotated[int, Query(ge=0)] = 0,
           limit: Annotated[int, Query(le=100)] = 20) -> dict:
    return {"skip": skip, "limit": limit}

Paging = Annotated[dict, Depends(paging)]     # reusable alias

@app.get("/todos")
def list_todos(page: Paging):
    return repo.list(**page)

@app.get("/users")
def list_users(page: Paging, q: str | None = None):
    return users.search(q, **page)

# /todos?skip=40&limit=10 → page = {"skip": 40, "limit": 10}
```

CODE EXPLAINED

- → `paging()` — normal function with query params
- → `Depends(paging)` — call it, inject the result
- → `Paging` alias — one name for type + dependency
- → Both routes get the same validated paging params

OTHER FRAMEWORK

- Flask — no DI; use `g`, decorators or helper functions
- Django — middleware, decorators, or class-based view mixins

PREFER / AVOID

- Avoid `page: dict = Depends(paging)` · Prefer `Annotated` aliases
- Avoid copying the same params into every route · Prefer one dependency

WHAT CAN BE A DEPENDENCY

KIND	EXAMPLE
function	def paging(…)
async function	async def get_user(…)
class	Depends(Pager) (sheet 22)
callable instance	Depends(RoleChecker("admin"))
generator	yield session (sheet 23)

COMMON USES

DEPENDENCY	PROVIDES
get_db	DB session
get_current_user	auth user
get_settings	config
paging	skip/limit
require_admin	permission check

⚠ COMMON MISTAKES

- Calling the function yourself: `Depends(paging())` passes the result, not the callable
- Using dependencies for heavy app-wide setup; put that in lifespan (sheet 26)
- Forgetting dependency params show in docs and leak internal names

✓ TIPS & BEST PRACTICES

- Keep dependencies in `deps.py` with `Annotated` aliases
- Return a small dataclass instead of a dict for type checking
- Override any dependency in tests (sheet 41)

QUICK SUMMARY

- `Depends(fn)` — FastAPI calls `fn`, injects result
- dependency params — resolved like route params
- `Annotated` alias — reuse everywhere
- per-request cache — runs once per request

Class & Sub-Dependencies

DESCRIPTION

A class works as a dependency because calling it creates an instance: its `__init__` parameters become request parameters. Dependencies can depend on other dependencies, forming **chains** such as session → current user → admin check. An **instance with `__call__`** lets you parameterize a dependency, like `RoleChecker("admin")`.

NOTES

- `Depends()` with no arg reuses the annotated type
- Sub-dependency results are cached per request too
- `use_cache=False` forces a fresh call
- Chains keep each dependency small and testable

CHAIN + PARAMETERIZED CHECK

```python
from dataclasses import dataclass
from typing import Annotated
from fastapi import Depends, HTTPException

@dataclass
class Pager:                                  # class dependency
    skip: int = 0
    limit: int = 20

def get_current_user(token: Annotated[str, Depends(oauth2)]) -> User:
    return decode_user(token)                 # sheet 33

CurrentUser = Annotated[User, Depends(get_current_user)]

class RoleChecker:
    def __init__(self, role: str):
        self.role = role
    def __call__(self, user: CurrentUser) -> User:
        if self.role not in user.roles:
            raise HTTPException(403, "Forbidden")
        return user

@app.get("/admin/users")
def admin_users(pager: Annotated[Pager, Depends()],
                admin: Annotated[User, Depends(RoleChecker("admin"))]):
    ...
```

CODE EXPLAINED

- → `Pager` dataclass — fields become query params
- → `Depends()` empty — uses the annotated class
- → `RoleChecker("admin")` — configured instance
- → `__call__(user: CurrentUser)` — sub-dependency chain
- → 403 raised before the route body runs

OTHER FRAMEWORK

- Flask — decorators like `@roles_required("admin")` (Flask-Security)
- Django — `PermissionRequiredMixin`, `@permission_required`

PREFER / AVOID

- Avoid copying role checks inside route bodies · Prefer a dependency
- Avoid one giant `get_everything` dependency · Prefer small chained ones

DEPENDENCY PATTERNS

PATTERN	USE
`class with __init__`	group params
`instance __call__`	configurable check
`chain A → B → C`	build on results
`Depends(use_cache=False)`	fresh value each time

RESOLUTION ORDER

STEP	WHAT HAPPENS
1	sub-dependencies resolved first
2	results cached by callable
3	dependency called
4	route called with all results

⚠ COMMON MISTAKES

- Mutable class-level defaults on dependency classes shared across requests
- Expecting `use_cache` to cache across requests: it is per request only
- Raising 401 for a logged-in user who lacks a role: use 403

✓ TIPS & BEST PRACTICES

- Name aliases after what they give: `CurrentUser`, `AdminUser`, `DbSession`
- Use dataclasses for parameter-group dependencies
- Put role checks on routers with `dependencies=[...]` (sheet 24)

QUICK SUMMARY

- class — `__init__` params become inputs
- chain — dependencies using dependencies
- `__call__` instance — parameterized dependency
- `Depends()` — infer from the annotation

Sheet 23

Dependencies with yield

yield setup/teardown · DB session per request · try / finally · exceptions after yield · cleanup order

DESCRIPTION

A dependency that **`yields`** gives the route a value, then resumes after the response is ready to run **cleanup**. That's the standard way to open a DB session per request and always close it. Exceptions raised in the route are re-raised at the `yield`, so you can roll back with `try/except` and clean up in `finally`.

NOTES

- Code before `yield` = setup; after = teardown
- Only one `yield` per dependency
- Cleanup runs in reverse order of setup
- Re-raise caught exceptions after `yield`, or they are swallowed

UNDER THE HOOD

- FastAPI wraps yield dependencies in an `AsyncExitStack` per request
- Exit code runs as the stack unwinds
- Teardown now runs before the response is sent, so cleanup can still affect it

SESSION PER REQUEST

```python
from typing import Annotated
from fastapi import Depends
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db                        # route runs here
        db.commit()
    except Exception:
        db.rollback()
        raise                           # keep the error
    finally:
        db.close()

DbSession = Annotated[Session, Depends(get_db)]

@app.post("/todos", status_code=201)
def create(todo: TodoIn, db: DbSession) -> TodoOut:
    row = Todo(**todo.model_dump())
    db.add(row)
    db.flush()                          # get row.id
    return row
```

CODE EXPLAINED

→ `yield db` — hands the session to the route

→ after `yield` — commit if the route succeeded

→ `except` + `raise` — roll back, keep the error

→ `finally` — always close

OTHER FRAMEWORK

- Flask — `teardown_appcontext` closes the session (sheet 53)
- Django — connections managed per request by the framework

PREFER / AVOID

- Avoid opening sessions inside each route · Prefer one yield dependency
- Avoid `except: pass` after `yield` · Prefer re-raising

LIFECYCLE

PHASE	RUNS
`setup`	before yield, before route
`route`	uses the yielded value
`teardown`	after route, in finally
`error`	re-raised at the yield

ASYNC VERSION

LINE	NOTE
`async def get_db():`	async generator
`async with Session() as s:`	context manager
`yield s`	same pattern
`await s.commit()`	async session (sheet 30)

⚠ COMMON MISTAKES

- Swallowing the exception after `yield`: client gets 200 and data isn't saved
- Using `yield` deps for app-wide resources: they run every request
- Committing in both the route and the dependency without knowing it

✓ TIPS & BEST PRACTICES

- Pick one place to commit (route or dependency) and stick to it
- Use `with` / `async with` inside the dependency for automatic close
- Use lifespan for engines and pools; yield deps for sessions

QUICK SUMMARY

- `yield` — setup, give value, teardown
- `try/except/finally` — rollback and close
- re-raise — never hide route errors
- session per request — the classic use

Global Dependencies & Overrides

DESCRIPTION

Dependencies don't have to be parameters. List them in `dependencies=[...]` on a route, a router or the whole app and they run for every matching request, with their return value ignored: perfect for **auth checks, API keys and rate limits**. `app.dependency_overrides` swaps any dependency for another, mainly in tests.

NOTES

- Order: app → router → route → parameter dependencies
- List form ignores return values
- Overrides are keyed by the original callable
- Clear overrides after each test

APPLY AT EVERY LEVEL

```python
from fastapi import APIRouter, Depends, FastAPI, Header, HTTPException

async def verify_key(x_api_key: str = Header()):
    if x_api_key != settings.api_key:
        raise HTTPException(401, "Bad API key")

app = FastAPI(dependencies=[Depends(log_request)])     # every route

admin = APIRouter(prefix="/admin",
                  dependencies=[Depends(RoleChecker("admin"))])

@app.get("/reports", dependencies=[Depends(verify_key)])
def reports(): ...                                     # just this one

app.include_router(admin)

# tests
def fake_user():
    return User(id=1, roles=["admin"])

app.dependency_overrides[get_current_user] = fake_user
# ... run tests ...
app.dependency_overrides.clear()
```

CODE EXPLAINED

- → `FastAPI(dependencies=)` — runs on all routes
- → `APIRouter(dependencies=)` — runs on that router's routes
- → decorator `dependencies=` — one route
- → `dependency_overrides[orig] = fake` — swap implementation
- → `.clear()` — restore real dependencies

OTHER FRAMEWORK

- Flask — `@bp.before_request` for blueprint-wide checks
- Django — middleware or `@method_decorator` on views

PREFER / AVOID

- Avoid unused params like `_: None = Depends(check)` · Prefer `dependencies=[...]`
- Avoid monkeypatching modules in tests · Prefer `dependency_overrides`

WHERE TO ATTACH

LEVEL	SCOPE
`FastAPI(dependencies=)`	whole app
`APIRouter(dependencies=)`	router
`include_router(..., dependencies=)`	that inclusion
`@app.get(dependencies=)`	single route

GOOD FITS

DEPENDENCY	LEVEL
`API key check`	router or app
`admin role`	admin router
`request logging`	app
`rate limit`	route or router
`feature flag`	route

⚠ COMMON MISTAKES

- Overriding with a function whose params differ, then getting surprise 422s
- Leaving overrides set between tests: later tests pass for the wrong reason
- Global auth dependency that also blocks `/health` and `/docs`

✓ TIPS & BEST PRACTICES

- Use a pytest fixture that sets and clears overrides
- Keep public routes on a separate router without the auth dependency
- Overrides work for sub-dependencies too

QUICK SUMMARY

- `dependencies=[...]` — run, ignore result
- app / router / route — pick the scope
- `dependency_overrides` — swap for tests
- clear overrides — avoid test leaks

APIRouter & Bigger Applications

DESCRIPTION

An **APIRouter** is a mini app: it has the same decorators as FastAPI and is mounted with `app.include_router()`. Give each feature its own router module with a `prefix`, `tags` and shared `dependencies`. Routers can include other routers, which makes **API versioning** (`/v1`, `/v2`) a matter of nesting.

NOTES

- Routers never need to import `app`
- `prefix` must start with `/` and not end with one
- `responses=` on a router documents shared errors
- Include order still decides overlapping matches

ROUTERS/TODOS.PY + MAIN.PY

```python
# app/routers/todos.py
from fastapi import APIRouter
from app.deps import CurrentUser, DbSession

router = APIRouter(prefix="/todos", tags=["todos"],
                   responses={404: {"description": "Not found"}})

@router.get("/")
def list_todos(user: CurrentUser, db: DbSession): ...

@router.get("/{todo_id}")
def get_todo(todo_id: int, user: CurrentUser): ...

# app/main.py
from fastapi import APIRouter, FastAPI
from app.routers import todos, users

v1 = APIRouter(prefix="/v1")
v1.include_router(todos.router)          # /v1/todos/...
v1.include_router(users.router)

app = FastAPI()
app.include_router(v1)
```

CODE EXPLAINED

- → `APIRouter(prefix=, tags=)` — shared settings
- → `@router.get("/")` — becomes `/todos/`
- → deps imported from `app.deps`, not `main`
- → `v1.include_router(...)` — nest for versions

OTHER FRAMEWORK

- Flask — `Blueprint("todos", __name__, url_prefix="/todos")` (sheet 51)
- Django — `include("todos.urls")` with `app_name`

PREFER / AVOID

- Avoid `from app.main import app` in routers · Prefer `APIRouter`
- Avoid mixing versions in one router · Prefer one router tree per version

INCLUDE_ROUTER() ARGS

ARG	EFFECT
`prefix=`	extra path prefix
`tags=`	extra tags
`dependencies=`	extra deps
`responses=`	extra documented codes
`include_in_schema=False`	hide from docs

ROUTER LAYOUT

FILE	ROUTER
`routers/todos.py`	prefix /todos
`routers/users.py`	prefix /users
`routers/auth.py`	prefix /auth
`routers/admin.py`	prefix /admin + role dep

⚠ COMMON MISTAKES

- Prefix `/todos/` plus path `/` gives `/todos//` problems: no trailing slash on prefix
- Defining routes after `include_router` and expecting them included: inclusion copies routes at that moment
- Circular imports between routers and `main.py`

✓ TIPS & BEST PRACTICES

- Use `@router.get("")` if you want `/todos` with no trailing slash
- Put shared dependencies on the router, not each route
- Mirror router files in `tests/`

QUICK SUMMARY

- `APIRouter` — mini app per feature
- `include_router()` — mount with prefix/tags/deps
- nested routers — `/v1`, `/v2`
- no `app` imports — no circular imports

Lifespan Events

DESCRIPTION

The **lifespan** function runs once when the app starts and once when it stops. Code before `yield` is **startup** (open DB pools, load ML models, create HTTP clients); code after is **shutdown** (close them). Store shared objects on `app.state` or yield a dict that becomes `request.state`.

NOTES

- Replaces `@app.on_event("startup")` / `("shutdown")`
- Runs per worker process, not once per server
- Startup failure stops the server from starting
- `TestClient` runs lifespan only inside `with TestClient(app)`

UNDER THE HOOD

- ASGI servers send `lifespan.startup` / `lifespan.shutdown` messages
- Starlette runs your context manager on those messages
- A yielded dict becomes the initial `request.state` for every request

SHARED RESOURCES

```python
from contextlib import asynccontextmanager
import httpx
from fastapi import FastAPI, Request

@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup
    engine = create_async_engine(settings.database_url)
    client = httpx.AsyncClient(timeout=10)
    model = load_model("sentiment.bin")
    yield {"client": client, "model": model}
    # shutdown
    await client.aclose()
    await engine.dispose()

app = FastAPI(lifespan=lifespan)

@app.get("/score")
async def score(text: str, request: Request):
    model = request.state.model
    return {"score": model.predict(text)}
```

CODE EXPLAINED

- → `@asynccontextmanager` — turns the generator into a lifespan
- → before `yield` — startup work
- → `yield {...}` — state shared with requests
- → after `yield` — shutdown cleanup

OTHER FRAMEWORK

- Flask — create resources in `create_app()`; no shutdown hook
- Django — `AppConfig.ready()` for startup code

PREFER / AVOID

- Avoid `@app.on_event` · Prefer `lifespan=`
- Avoid creating `httpx.AsyncClient()` per request · Prefer one in lifespan

GOOD LIFESPAN JOBS

STARTUP	SHUTDOWN
create DB engine/pool	engine.dispose()
open HTTP client	client.aclose()
load ML model	free memory
connect Redis	close connection
validate settings	—

WHERE STATE LIVES

OPTION	ACCESS
yield dict	request.state.key
app.state.key = x	request.app.state.key
module global	import (avoid)
dependency	Depends(get_x)

⚠ COMMON MISTAKES

- Using a plain `TestClient(app)` without `with`: startup never runs
- Heavy startup in every worker multiplies memory use
- Mixing `lifespan=` with `on_event` handlers: the event handlers are ignored

✓ TIPS & BEST PRACTICES

- Wrap state access in a dependency so tests can override it
- Fail fast: check DB connectivity during startup
- Log startup and shutdown so deploys are easy to debug

QUICK SUMMARY

- `lifespan` — startup before `yield`, shutdown after
- `yield {...}` — becomes `request.state`
- per worker — runs in each process
- replaces `on_event`

Middleware & CORS

DESCRIPTION

Middleware wraps every request and response: it can time requests, add headers, or reject traffic before routes run. Write one with `@app.middleware("http")` or add a class with `app.add_middleware()`. **CORS** middleware is what lets a browser app on another origin call your API.

NOTES

- Last added = outermost = runs first
- Middleware sees all routes, including 404s
- CORS is enforced by browsers, not servers or curl
- Exceptions in routes reach middleware as 500 responses

UNDER THE HOOD

- Stack: `ServerErrorMiddleware` outermost, `ExceptionMiddleware` innermost
- `@app.middleware` uses `BaseHTTPMiddleware`; pure ASGI middleware is faster
- Preflight `OPTIONS` requests are answered by `CORSMiddleware` itself

TIMING HEADER + CORS

```python
import time
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
@app.middleware("http")
async def add_timing(request: Request, call_next):
    start = time.perf_counter()
    response = await call_next(request)          # run the route
    ms = (time.perf_counter() - start) * 1000
    response.headers["Server-Timing"] = f"app;dur={ms:.1f}"
    return response

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com", "http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CODE EXPLAINED

- → `call_next(request)` — passes control inward
- → code after `call_next` — edits the response
- → `allow_origins` — exact origins, scheme + host + port

OTHER FRAMEWORK

- Flask — `flask-cors` and `before_request` / `after_request` hooks
- Django — `MIDDLEWARE` list + `django-cors-headers`

PREFER / AVOID

- Avoid `allow_origins=["*"]` with credentials · Prefer an explicit list
- Avoid auth checks in middleware · Prefer dependencies (per route, testable)

BUILT-IN MIDDLEWARE

CLASS	DOES
`CORSMiddleware`	cross-origin headers
`GZipMiddleware`	compress responses
`TrustedHostMiddleware`	reject bad Host headers
`HTTPSRedirectMiddleware`	http → https
`SessionMiddleware`	signed cookie sessions

CORSMIDDLEWARE OPTIONS

OPTION	MEANING
`allow_origins`	allowed origins list
`allow_origin_regex`	pattern instead
`allow_credentials`	cookies / auth headers
`allow_methods`	methods
`expose_headers`	headers JS can read
`max_age`	preflight cache secs

⚠ COMMON MISTAKES

- `allow_origins=["*"]` + `allow_credentials=True`: browsers reject it
- Trailing slash in an origin (`https://app.com/`): never matches
- Reading `await request.body()` in `BaseHTTPMiddleware` and breaking streaming

✓ TIPS & BEST PRACTICES

- Add CORS last so it wraps everything, including errors
- Use `TrustedHostMiddleware` in production
- Put request IDs in middleware and in log context

QUICK SUMMARY

- `@app.middleware("http")` — wrap every request
- `add_middleware()` — add built-in classes
- `CORSMiddleware` — browser cross-origin access
- last added — runs first

SQLModel

DESCRIPTION

SQLModel (by FastAPI's author) merges Pydantic and SQLAlchemy: one class can be both a validated model and a database table. Use `table=True` for real tables and plain `SQLModel` classes for API schemas that inherit shared fields. Under the hood it is SQLAlchemy, so its sessions and `select()` work as usual.

NOTES

- `pip install sqlmodel`
- `table=True` classes are tables; others are just schemas
- `create_all()` is fine for demos; use Alembic in real apps
- SQLite needs `check_same_thread=False` with FastAPI

TABLE + SCHEMAS + ROUTE

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select

class TodoBase(SQLModel):
    title: str = Field(index=True, max_length=100)
    done: bool = False

class Todo(TodoBase, table=True):            # DB table
    id: int | None = Field(default=None, primary_key=True)
    owner_id: int = Field(foreign_key="user.id")

class TodoCreate(TodoBase): pass             # API input
class TodoRead(TodoBase): id: int            # API output

engine = create_engine("sqlite:///todos.db",
                       connect_args={"check_same_thread": False})
SQLModel.metadata.create_all(engine)

@app.get("/todos", response_model=list[TodoRead])
def list_todos(done: bool | None = None):
    with Session(engine) as s:
        stmt = select(Todo)
        if done is not None:
            stmt = stmt.where(Todo.done == done)
        return s.exec(stmt).all()
```

CODE EXPLAINED

- → `TodoBase` — shared fields, no table
- → `table=True` — real table; `id` is the primary key
- → `TodoCreate` / `TodoRead` — input and output schemas
- → `select(Todo).where(...)` — typed query
- → `s.exec(stmt).all()` — run and fetch rows

OTHER FRAMEWORK

- Flask — Flask-SQLAlchemy `db.Model` (sheet 54)
- Django — `models.Model` with the Django ORM

PREFER / AVOID

- Avoid returning `table=True` models with secret columns · Prefer a `Read` schema
- Avoid `create_all()` in production · Prefer Alembic migrations

FIELD() FOR TABLES

ARG	EFFECT
`primary_key=True`	PK column
`foreign_key="user.id"`	FK
`index=True`	create index
`unique=True`	unique constraint
`nullable=False`	NOT NULL
`sa_column=`	raw SQLAlchemy column

SESSION CALLS

CALL	DOES
`s.add(obj)`	stage insert/update
`s.commit()`	write
`s.refresh(obj)`	reload (get id)
`s.get(Todo, 7)`	by primary key
`s.exec(select(...))`	query
`s.delete(obj)`	stage delete

⚠ COMMON MISTAKES

- Expecting `table=True` models to validate on construction: they skip validation for speed
- Forgetting `s.refresh(obj)` and returning an object with `id=None`
- Sharing one `Session` across requests

✓ TIPS & BEST PRACTICES

- Provide the session through a yield dependency (sheet 23)
- Use `TodoUpdate` with all-optional fields for PATCH
- Drop to SQLAlchemy when SQLModel lacks a feature; both mix fine

QUICK SUMMARY

- `SQLModel, table=True` — model + table
- Base / Create / Read — shared + API schemas
- `Session` + `select()` — SQLAlchemy underneath
- Alembic — migrations for real apps

SQLAlchemy 2.0

DESCRIPTION

SQLAlchemy 2.0 is the standard Python ORM and works with FastAPI directly. Models use typed `Mapped[...]` annotations, queries use `select()`, and sessions come from a `sessionmaker`. Keep ORM models separate from Pydantic schemas and convert with `from_attributes=True`.

NOTES

- `pip install sqlalchemy` (+ a driver like `psycopg`)
- 2.0 style: `select()` everywhere, no `session.query()`
- `Mapped[str | None]` → nullable column
- `expire_on_commit=False` keeps objects usable after commit

MODELS, SESSION, QUERY

```python
from sqlalchemy import ForeignKey, String, create_engine, select
from sqlalchemy.orm import (DeclarativeBase, Mapped, mapped_column,
                            relationship, sessionmaker)

class Base(DeclarativeBase): pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True)
    todos: Mapped[list["Todo"]] = relationship(back_populates="owner")

class Todo(Base):
    __tablename__ = "todos"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
    owner: Mapped[User] = relationship(back_populates="todos")

engine = create_engine("postgresql+psycopg://app@localhost/app")
SessionLocal = sessionmaker(engine, expire_on_commit=False)

def todos_for(db, user_id: int) -> list[Todo]:
    return db.scalars(select(Todo).where(Todo.owner_id == user_id)).all()
```

CODE EXPLAINED

- → `DeclarativeBase` — base class for models
- → `Mapped[int]` + `mapped_column()` — typed column
- → `relationship(back_populates=)` — two-way link
- → `sessionmaker(engine)` — session factory
- → `db.scalars(select(...))` — ORM objects, not rows

OTHER FRAMEWORK

- Flask — Flask-SQLAlchemy wraps the same models (sheet 54)
- Django — Django ORM: `Todo.objects.filter(owner_id=...)`

PREFER / AVOID

- Avoid legacy `session.query(Todo)` · Prefer `select(Todo)`
- Avoid lazy-loading in loops (N+1) · Prefer `selectinload(User.todos)`

QUERY CHEATS

2.0 CODE	DOES
`db.get(Todo, 7)`	by primary key
`select(Todo).where(...)`	filter
`.order_by(Todo.id.desc())`	sort
`.limit(20).offset(40)`	page
`.options(selectinload(...))`	eager load
`db.scalar(select(func.count()))`	count

DRIVERS

DATABASE	URL PREFIX
PostgreSQL	postgresql+psycopg://
PostgreSQL async	postgresql+asyncpg://
MySQL	mysql+pymysql://
SQLite	sqlite:///file.db
SQLite async	sqlite+aiosqlite://

⚠ COMMON MISTAKES

- Returning ORM objects whose lazy relationships load after the session closed
- Forgetting `String(n)` length on MySQL
- Creating a new engine per request instead of one per app

✓ TIPS & BEST PRACTICES

- Use a yield dependency for sessions (sheet 23)
- Turn on `echo=True` on the engine to see SQL while learning
- See the SQL set for joins and indexes

QUICK SUMMARY

- `Mapped[...]` — typed ORM columns
- `select()` — 2.0 query style
- `sessionmaker` — session per request
- `relationship()` — linked objects

Async Database Access

DESCRIPTION

With `async def` routes, use SQLAlchemy's **async** API so database waits don't block the event loop. You need an async driver (`asyncpg`, `aiosqlite`, `psycopg` async), an `AsyncEngine`, and `AsyncSession`. Every query is `await`ed, and lazy loading is off-limits, so load relationships eagerly.

NOTES

- Async only helps if every I/O call in the route is async
- Lazy relationship access raises `MissingGreenlet`
- Use `selectinload()` for relationships
- Create the engine in lifespan, dispose on shutdown

ASYNC SESSION DEPENDENCY

```python
from typing import Annotated, AsyncIterator
from fastapi import Depends
from sqlalchemy import select
from sqlalchemy.ext.asyncio import (AsyncSession,
    async_sessionmaker, create_async_engine)
from sqlalchemy.orm import selectinload

engine = create_async_engine(
    "postgresql+asyncpg://app@localhost/app", pool_size=10)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False)

async def get_db() -> AsyncIterator[AsyncSession]:
    async with SessionLocal() as session:
        yield session

Db = Annotated[AsyncSession, Depends(get_db)]

@app.get("/users/{user_id}")
async def get_user(user_id: int, db: Db) -> UserWithTodos:
    stmt = (select(User).where(User.id == user_id)
            .options(selectinload(User.todos)))
    user = await db.scalar(stmt)
    if user is None:
        raise HTTPException(404)
    return user
```

CODE EXPLAINED

- → `create_async_engine` — async driver URL
- → `async_sessionmaker` — `AsyncSession` factory
- → `async with` — closes the session for you
- → `selectinload` — load todos up front
- → `await db.scalar(...)` — one object or `None`

OTHER FRAMEWORK

- Flask — sync SQLAlchemy only (WSGI); async views gain little here
- Django — async ORM: `await Todo.objects.aget(pk=1)`

PREFER / AVOID

- Avoid sync drivers in `async def` routes · Prefer an async driver
- Avoid `user.todos` lazy access · Prefer `selectinload` or `await user.awaitable_attrs.todos`

SYNC → ASYNC

SYNC	ASYNC
`create_engine`	create_async_engine
`Session`	AsyncSession
`db.scalars(q)`	await db.scalars(q)
`db.commit()`	await db.commit()
`db.get(T, 1)`	await db.get(T, 1)
`engine.dispose()`	await engine.dispose()

ASYNC DRIVERS

DATABASE	PACKAGE
PostgreSQL	asyncpg or psycopg[binary]
MySQL	aiomysql / asyncmy
SQLite	aiosqlite
MongoDB	motor / pymongo async (not SQLAlchemy)

⚠ COMMON MISTAKES

- `MissingGreenlet` errors from lazy loads after the query
- Mixing a sync `Session` into async routes: blocks the loop
- `expire_on_commit=True` (default): attributes expire, then need a lazy reload

✓ TIPS & BEST PRACTICES

- Return Pydantic models built inside the session to avoid lazy loads
- Use `pool_pre_ping=True` for long-lived connections
- Plain `def` routes + sync SQLAlchemy is a fine, simpler choice

QUICK SUMMARY

- `create_async_engine` + async driver
- `AsyncSession` — `await` every call
- `selectinload` — no lazy loads
- yield dependency — session per request

Alembic Migrations & CRUD Patterns

DESCRIPTION

Alembic tracks schema changes as versioned migration scripts, so every environment's database can be brought to the same state. Point `env.py` at your models' metadata and `--autogenerate` drafts migrations from model changes. Keep data access in small **CRUD functions** so routes stay thin.

NOTES

- Always read autogenerated migrations before running them
- Autogenerate misses some changes (renames, some constraints)
- Run `alembic upgrade head` in deploys, not `create_all()`
- Commit the `alembic/versions/` folder

ALEMBIC WORKFLOW

```
alembic init alembic                      # creates alembic/ + alembic.ini
# alembic/env.py:  target_metadata = Base.metadata
alembic revision --autogenerate -m "add todos.due"
alembic upgrade head                      # apply all
alembic downgrade -1                      # undo one
alembic current                           # where am I?
```

CRUD.PY — THIN DATA LAYER

```
def get_todo(db: Session, todo_id: int) -> Todo | None:
    return db.get(Todo, todo_id)

def update_todo(db: Session, todo: Todo, data: TodoUpdate) -> Todo:
    for key, value in data.model_dump(exclude_unset=True).items():
        setattr(todo, key, value)          # only fields sent
    db.add(todo)
    db.commit()
    db.refresh(todo)
    return todo
```

CODE EXPLAINED

- → `alembic init` — scaffolding + config
- → `target_metadata` — lets autogenerate compare models
- → `--autogenerate -m` — draft a migration
- → `exclude_unset=True` — PATCH only changes sent fields
- → `refresh()` — reload DB defaults

OTHER FRAMEWORK

- Flask — Flask-Migrate wraps Alembic: `flask db migrate` (sheet 54)
- Django — `makemigrations` / `migrate` built in

PREFER / AVOID

- Avoid editing a migration after it ran elsewhere · Prefer a new migration
- Avoid SQL in route bodies · Prefer `crud.py` or repository functions

ALEMBIC COMMANDS

COMMAND	DOES
`revision -m "msg"`	empty migration
`revision --autogenerate`	diff models → DB
`upgrade head / +1`	apply
`downgrade -1 / base`	revert
`history`	list revisions
`stamp head`	mark as current, no SQL

MIGRATION OPS

OP.	DOES
`create_table`	new table
`add_column`	new column
`alter_column`	type/nullability
`create_index`	index
`execute("SQL")`	raw SQL / data fix

⚠ COMMON MISTAKES

- Not setting `target_metadata`: autogenerate produces empty migrations
- Adding a NOT NULL column without a default to a table with rows
- Two branches creating migrations from the same head: merge with `alembic merge`

✓ TIPS & BEST PRACTICES

- Use `alembic init -t async alembic` for async engines
- Read the DB URL from settings inside `env.py`
- Test `downgrade` once so rollbacks are possible

QUICK SUMMARY

- `revision --autogenerate` — draft migration
- `upgrade head` — apply all
- CRUD functions — thin routes
- `exclude_unset` — true PATCH

Security Basics & OAuth2 Password Flow

DESCRIPTION

`fastapi.security` provides dependency classes that read credentials and document them in OpenAPI. **`OAuth2PasswordBearer`** reads `Authorization: Bearer <token>` and returns the token string. **`OAuth2PasswordRequestForm`** parses the username/password form a login endpoint receives. Together they light up the **Authorize** button in `/docs`.

NOTES

- These classes only extract credentials; you verify them
- Missing header → automatic 401 with `WWW-Authenticate`
- `tokenUrl` is relative and used by `/docs` to log in
- The login form is form-encoded, not JSON

LOGIN + PROTECTED ROUTE

```python
from typing import Annotated
from fastapi import Depends, HTTPException, status
from fastapi.security import (OAuth2PasswordBearer,
                              OAuth2PasswordRequestForm)

oauth2 = OAuth2PasswordBearer(tokenUrl="auth/token")

@app.post("/auth/token")
def login(form: Annotated[OAuth2PasswordRequestForm, Depends()]):
    user = authenticate(form.username, form.password)
    if not user:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED,
                            "Incorrect username or password",
                            headers={"WWW-Authenticate": "Bearer"})
    return {"access_token": create_token(user.id),
            "token_type": "bearer"}

def get_current_user(token: Annotated[str, Depends(oauth2)]) -> User:
    return user_from_token(token)          # sheet 33

@app.get("/me")
def me(user: Annotated[User, Depends(get_current_user)]) -> UserOut:
    return user
```

CODE EXPLAINED

- → `OAuth2PasswordBearer(tokenUrl=)` — reads the Bearer token
- → `OAuth2PasswordRequestForm` — `username`, `password`, `scope`
- → Response must include `access_token` and `token_type`
- → `get_current_user` — turns a token into a user
- → `/me` — any route can require a user

OTHER FRAMEWORK

- Flask — Flask-Login sessions (sheet 55) or Flask-JWT-Extended
- Django — built-in auth + sessions; DRF token/JWT classes

PREFER / AVOID

- Avoid reading the header manually · Prefer `OAuth2PasswordBearer`
- Avoid saying which part was wrong · Prefer "Incorrect username or password"

SECURITY CLASSES

CLASS	READS
`OAuth2PasswordBearer`	Bearer token
`HTTPBearer`	Bearer, any scheme
`HTTPBasic`	Basic auth
`APIKeyHeader`	key in a header
`APIKeyQuery / APIKeyCookie`	key in query/cookie
`OpenIdConnect`	OIDC config URL

TOKEN RESPONSE

KEY	VALUE
`access_token`	the JWT
`token_type`	"bearer"
`expires_in`	seconds (optional)
`refresh_token`	optional (sheet 33)

⚠ COMMON MISTAKES

- Expecting JSON on `/token`: the OAuth2 spec uses form fields
- Forgetting `python-multipart`, so the form can't be parsed
- Returning 403 for a missing token; it should be 401

✓ TIPS & BEST PRACTICES

- Use a real identity provider (Auth0, Cognito, Keycloak) for big apps
- Rate-limit the login endpoint
- Use the Authorize button in `/docs` to test protected routes

QUICK SUMMARY

- `OAuth2PasswordBearer` — extract Bearer token
- `OAuth2PasswordRequestForm` — login form
- `/token` returns `access_token` + `token_type`
- `get_current_user` — token → user dependency

JWT Authentication

DESCRIPTION

A **JWT** is a signed token carrying claims like the user ID (`sub`) and expiry (`exp`). The server signs it at login and verifies the signature on each request, so no session lookup is needed. Use **PyJWT** to encode and decode, keep access tokens short-lived, and use refresh tokens to get new ones.

NOTES

- `pip install pyjwt`
- JWT payloads are signed, **not encrypted**: no secrets inside
- `decode` checks `exp` automatically
- HS256 = shared secret; RS256 = public/private key pair

CREATE & VERIFY TOKENS

```python
from datetime import datetime, timedelta, timezone
import jwt
from jwt.exceptions import InvalidTokenError

SECRET = settings.secret_key            # 32+ random bytes
ALGO = "HS256"

def create_token(user_id: int, minutes: int = 15) -> str:
    now = datetime.now(timezone.utc)
    claims = {"sub": str(user_id), "iat": now,
              "exp": now + timedelta(minutes=minutes)}
    return jwt.encode(claims, SECRET, algorithm=ALGO)

def get_current_user(token: Annotated[str, Depends(oauth2)],
                     db: DbSession) -> User:
    unauthorized = HTTPException(401, "Could not validate credentials",
                                 headers={"WWW-Authenticate": "Bearer"})
    try:
        payload = jwt.decode(token, SECRET, algorithms=[ALGO])
    except InvalidTokenError:           # bad signature, expired...
        raise unauthorized
    user = db.get(User, int(payload["sub"]))
    if user is None or not user.is_active:
        raise unauthorized
    return user
```

CODE EXPLAINED

- → `sub` — subject (user ID), must be a string
- → `exp` — expiry; PyJWT rejects expired tokens
- → `algorithms=[ALGO]` — always pin allowed algorithms
- → `InvalidTokenError` — base for all token errors
- → Same 401 for every failure

OTHER FRAMEWORK

- Flask — Flask-JWT-Extended: `create_access_token()`, `@jwt_required()`
- Django — `djangorestframework-simplejwt`

PREFER / AVOID

- Avoid long-lived access tokens · Prefer ~15 min + refresh tokens
- Avoid `algorithms` omitted or `"none"` · Prefer an explicit list

STANDARD CLAIMS

CLAIM	MEANING
`sub`	user ID
`exp`	expires at
`iat`	issued at
`nbf`	not before
`iss / aud`	issuer / audience
`jti`	token ID (for revocation)

TOKEN STRATEGY

TOKEN	TYPICAL LIFE
`access`	5–15 minutes
`refresh`	days, rotate on use
`revocation`	deny-list by jti
`storage (browser)`	httpOnly cookie > localStorage

⚠ COMMON MISTAKES

- Putting emails or roles you rely on for security in an unverifiable place
- Hard-coding the secret in source control
- Integer `sub`: spec says string; PyJWT may reject it

✓ TIPS & BEST PRACTICES

- Generate secrets with `openssl rand -hex 32`
- Add `aud` and `iss` checks if several services share keys
- Use RS256 when other services must verify but not issue tokens

QUICK SUMMARY

- JWT — signed claims, not encrypted
- `jwt.encode/decode` — PyJWT
- `exp` — short-lived access tokens
- one 401 for all auth failures

Sheet 34

Password Hashing & Users

pwdlib (Argon2) · hash / verify · never store plain text · get_current_active_user · user models

DESCRIPTION

Never store passwords: store a **slow, salted hash** and compare on login. FastAPI's docs now recommend **pwdlib** with **Argon2**, the current best-practice algorithm. Keep three user shapes: `UserCreate` (has password), the DB `User` (has `hashed_password`), and `UserOut` (has neither).

NOTES

- `pip install "pwdlib[argon2]"`
- Each hash includes its own salt and settings
- `verify` takes constant time; don't compare strings yourself
- passlib is unmaintained; prefer pwdlib

REGISTER & AUTHENTICATE

```python
from pwdlib import PasswordHash

hasher = PasswordHash.recommended()        # Argon2id

def hash_password(raw: str) -> str:
    return hasher.hash(raw)                # "$argon2id$v=19$m=65536,..."

def authenticate(db, email: str, password: str) -> User | None:
    user = db.scalar(select(User).where(User.email == email))
    if user is None:
        hasher.hash(password)              # same work, no timing leak
        return None
    if not hasher.verify(password, user.hashed_password):
        return None
    return user

@app.post("/users", status_code=201)
def register(data: UserCreate, db: DbSession) -> UserOut:
    if db.scalar(select(User).where(User.email == data.email)):
        raise HTTPException(409, "Email already registered")
    user = User(email=data.email,
                hashed_password=hash_password(data.password))
    db.add(user); db.commit(); db.refresh(user)
    return user
```

CODE EXPLAINED

- → `PasswordHash.recommended()` — Argon2id with safe defaults
- → `hasher.hash()` — salt + hash in one string
- → `hasher.verify()` — check a login attempt
- → dummy hash on unknown email — equal timing
- → 409 — duplicate email

OTHER FRAMEWORK

- Flask — `werkzeug.security.generate_password_hash()` / `check_password_hash()`
- Django — `user.set_password()` / `check_password()` (PBKDF2 or Argon2)

PREFER / AVOID

- Avoid MD5/SHA-256 for passwords · Prefer Argon2 or bcrypt
- Avoid returning the DB `User` · Prefer `UserOut`

USER SHAPES

MODEL	FIELDS
`UserCreate`	email, password
`User (DB)`	id, email, hashed_password, is_active
`UserOut`	id, email
`UserUpdate`	all optional

HASH ALGORITHMS

ALGORITHM	STATUS
`Argon2id`	recommended
`bcrypt`	fine, 72-byte limit
`scrypt`	fine
`PBKDF2`	OK with high iterations
`MD5 / SHA-1 / SHA-256`	never for passwords

⚠ COMMON MISTAKES

- Logging request bodies that include passwords
- Different error messages for "no user" vs "wrong password"
- Enforcing maximum lengths too small; allow long passphrases

✓ TIPS & BEST PRACTICES

- Use `hasher.verify_and_update()` to upgrade old hashes on login
- Add `is_active` and check it in `get_current_user`
- Require `min_length=12` on `UserCreate.password`

QUICK SUMMARY

- pwdlib + Argon2 — current best practice
- `hash()` / `verify()` — never compare yourself
- three user models — create, DB, out
- same timing — unknown user vs wrong password

Scopes, API Keys & HTTP Basic

Security(scopes=) · SecurityScopes · APIKeyHeader · HTTPBasic · HTTPBearer · secrets.compare_digest

DESCRIPTION

Beyond user logins, APIs often need **scopes** (fine-grained permissions inside a token), **API keys** for machine clients, and occasionally **HTTP Basic** for simple internal tools. `Security()` is `Depends()` plus a list of required scopes, and `SecurityScopes` tells the dependency which scopes the route needs.

NOTES

- Scopes are strings like `todos:read`, `todos:write`
- Declare available scopes on `OAuth2PasswordBearer(scopes={...})`
- API keys: send in a header, never in URLs (they get logged)
- Compare secrets with `secrets.compare_digest`

SCOPES + API KEY

```python
import secrets
from fastapi import Security
from fastapi.security import APIKeyHeader, SecurityScopes

oauth2 = OAuth2PasswordBearer(tokenUrl="auth/token",
    scopes={"todos:read": "Read todos", "todos:write": "Edit todos"})

def current_user(scopes: SecurityScopes,
                 token: Annotated[str, Depends(oauth2)]) -> User:
    payload = decode(token)                    # sheet 33
    granted = set(payload.get("scopes", []))
    if not set(scopes.scopes) <= granted:
        raise HTTPException(403, "Not enough permissions")
    return load_user(payload)

@app.post("/todos")
def create(user: Annotated[User, Security(current_user,
                                          scopes=["todos:write"])]): ...

api_key = APIKeyHeader(name="X-API-Key")

def require_key(key: Annotated[str, Depends(api_key)]):
    if not secrets.compare_digest(key, settings.api_key):
        raise HTTPException(401, "Invalid API key")
```

CODE EXPLAINED

- → `scopes={...}` — documented scopes for `/docs`
- → `SecurityScopes.scopes` — scopes this route requires
- → `Security(dep, scopes=[...])` — require scopes
- → `APIKeyHeader(name=)` — reads `X-API-Key`
- → `compare_digest` — constant-time comparison

OTHER FRAMEWORK

- Flask — custom decorator checking `request.headers["X-API-Key"]`
- Django — DRF permissions; `djangorestframework-api-key`

PREFER / AVOID

- Avoid API keys in query strings · Prefer a header
- Avoid `key == expected` · Prefer `secrets.compare_digest`

CHOOSING AN AUTH METHOD

METHOD	BEST FOR
OAuth2 + JWT	users, SPAs, mobile
Scopes	per-feature permissions
API key	server-to-server
HTTP Basic	internal tools over HTTPS
Session cookie	server-rendered sites

SECURITY HELPERS

NAME	GIVES
HTTPBasic()	HTTPBasicCredentials(username, password)
HTTPBearer()	HTTPAuthorizationCredentials
APIKeyHeader(name=)	key string
SecurityScopes	.scopes, .scope_str

⚠ COMMON MISTAKES

- Storing API keys in plain text in the DB: hash them like passwords
- Using 401 when a valid user lacks a scope: that's 403
- HTTP Basic over plain HTTP: credentials are only base64

✓ TIPS & BEST PRACTICES

- Prefix keys (`tk_live_...`) so leaks are easy to spot and scan for
- Let users create and revoke multiple keys
- Put scope checks on routers for whole feature areas

QUICK SUMMARY

- `Security(dep, scopes=)` — require permissions
- `SecurityScopes` — route's needed scopes
- `APIKeyHeader` — machine clients
- `compare_digest` — safe secret comparison

async def vs def

DESCRIPTION

FastAPI runs **`async def`** routes on the event loop and **plain `def`** routes in a **threadpool**. Both are fast when used right. The one real mistake is calling **blocking** code (`requests`, `time.sleep`, sync DB drivers) inside `async def`: that freezes every other request on that worker.

NOTES

- Rule: `async def` only if everything inside is awaitable
- Not sure? Use `def`; it's safe
- Same rules apply to dependencies
- CPU-heavy work belongs in a process pool or task queue

UNDER THE HOOD

- The threadpool comes from AnyIO, default 40 threads
- `def` routes are run with `run_in_threadpool`
- One blocking call in `async def` stalls the whole event loop

RIGHT AND WRONG

```python
import time, asyncio, httpx, requests
from fastapi.concurrency import run_in_threadpool

@app.get("/bad")
async def bad():
    time.sleep(2)                          # blocks the event loop!
    return requests.get(URL).json()        # also blocking

@app.get("/good-sync")
def good_sync():
    time.sleep(2)                          # fine: runs in a thread
    return requests.get(URL).json()

@app.get("/good-async")
async def good_async(request: Request):
    await asyncio.sleep(2)                 # yields to other requests
    client: httpx.AsyncClient = request.state.client
    r1, r2 = await asyncio.gather(client.get(URL_A), client.get(URL_B))
    return {"a": r1.json(), "b": r2.json()}

@app.get("/mixed")
async def mixed():
    data = await run_in_threadpool(legacy_sync_call, 42)
    return data
```

CODE EXPLAINED

- → `/bad` — `time.sleep` and `requests` freeze the worker
- → `/good-sync` — plain `def` runs in the threadpool
- → `asyncio.gather` — run two calls at once
- → `run_in_threadpool` — call sync code from async

OTHER FRAMEWORK

- Flask — WSGI: one request per thread; `async def` views run in a loop per request
- Django — async views + `sync_to_async()` for sync code

PREFER / AVOID

- Avoid `requests` in `async def` · Prefer `httpx.AsyncClient`
- Avoid `async def` by default · Prefer it only when you `await` something

WHICH TO USE

INSIDE THE ROUTE	DECLARE AS
async DB/HTTP calls	async def
sync DB driver / requests	def
pure CPU, quick	either
CPU heavy (seconds)	task queue / process pool
mix of both	async def + run_in_threadpool

BLOCKING → ASYNC

BLOCKING	ASYNC
time.sleep	asyncio.sleep
requests	httpx.AsyncClient
psycopg2	asyncpg / psycopg async
open().read()	aiofiles / anyio.Path
redis	redis.asyncio

⚠ COMMON MISTAKES

- Declaring everything `async def` out of habit while using sync libraries
- Running CPU-heavy model inference in `async def`: blocks all requests
- Creating a new `httpx.AsyncClient` per request

✓ TIPS & BEST PRACTICES

- Load-test with `hey` or `locust` before deciding async helps
- Use `asyncio.timeout()` around slow calls
- Raise the threadpool size only after measuring

QUICK SUMMARY

- `async def` — event loop; never block
- `def` — threadpool; safe default
- `gather` — parallel awaits
- `run_in_threadpool` — sync from async

Background Tasks

DESCRIPTION

BackgroundTasks runs small jobs **after the response is sent**, in the same process: sending a welcome email, writing an audit log, warming a cache. Declare a `BackgroundTasks` parameter and call `add_task()`. For anything slow, retryable or important, use a real **task queue** instead.

NOTES

- Tasks run after the response, in order added
- `def` tasks run in the threadpool; `async def` on the loop
- No retries, no persistence: a crash loses the task
- Dependencies can add tasks too

AFTER-RESPONSE WORK

```python
from fastapi import BackgroundTasks

def send_welcome(email: str, name: str):
    mailer.send(to=email, subject="Welcome!",
                body=f"Hi {name}, thanks for signing up.")

def write_audit(event: str, user_id: int):
    audit_log.append(event=event, user_id=user_id)

@app.post("/users", status_code=201)
def register(data: UserCreate, tasks: BackgroundTasks,
             db: DbSession) -> UserOut:
    user = create_user(db, data)
    tasks.add_task(send_welcome, user.email, name=user.name)
    tasks.add_task(write_audit, "register", user.id)
    return user                          # client gets this right away

def audit_dep(tasks: BackgroundTasks, request: Request):
    tasks.add_task(write_audit, f"hit {request.url.path}", 0)
```

CODE EXPLAINED

- → `tasks: BackgroundTasks` — injected per request
- → `add_task(fn, *args, **kwargs)` — schedule work
- → Response is sent before tasks run
- → `audit_dep` — a dependency can add tasks as well

OTHER FRAMEWORK

- Flask — no built-in; use Celery, RQ or a thread
- Django — `django.tasks` (6.0+) or Celery

PREFER / AVOID

- Avoid long jobs in `BackgroundTasks` · Prefer Celery, ARQ, RQ or Dramatiq
- Avoid passing ORM objects to tasks · Prefer IDs and reload

WHEN TO USE WHAT

JOB	TOOL
email, log, < 1 s	BackgroundTasks
must not be lost	task queue
needs retries	task queue
scheduled (cron)	Celery beat / APScheduler
CPU heavy	worker processes

TASK QUEUES

LIBRARY	NOTES
Celery	mature, Redis/RabbitMQ
ARQ	asyncio + Redis
RQ	simple, Redis
Dramatiq	simple, reliable
Taskiq	async-first

⚠ COMMON MISTAKES

- Using the request's DB session inside a task: it's closed by then
- Expecting task errors to reach the client: the response is already sent
- Running minutes-long work: holds a worker and dies on deploy

✓ TIPS & BEST PRACTICES

- Open a fresh session inside the task
- Log task failures; nobody else will see them
- Move to a queue as soon as you need retries

QUICK SUMMARY

- `BackgroundTasks` — run after response
- `add_task(fn, ...)` — schedule
- same process — no retries, no persistence
- task queue — for anything important

WebSockets

DESCRIPTION

WebSockets keep a two-way connection open for real-time features like chat, live dashboards and notifications. Declare an endpoint with `@app.websocket`, `accept()` the connection, then loop on `receive_*` and `send_*`. A disconnect raises **`WebSocketDisconnect`**, which you catch to clean up.

NOTES

- URL scheme is `ws://` or `wss://` (TLS)
- Dependencies, path and query params work on websockets
- Multiple workers don't share connections; use Redis pub/sub
- `TestClient.websocket_connect()` tests them

CHAT ROOM

```python
from fastapi import WebSocket, WebSocketDisconnect

class Manager:
    def __init__(self):
        self.active: list[WebSocket] = []
    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.active.append(ws)
    def disconnect(self, ws: WebSocket):
        self.active.remove(ws)
    async def broadcast(self, msg: dict):
        for ws in self.active:
            await ws.send_json(msg)

manager = Manager()

@app.websocket("/ws/{room}")
async def chat(ws: WebSocket, room: str, name: str = "anon"):
    await manager.connect(ws)
    try:
        while True:
            text = await ws.receive_text()
            await manager.broadcast(
                {"room": room, "from": name, "text": text})
    except WebSocketDisconnect:
        manager.disconnect(ws)
```

CODE EXPLAINED

- → `@app.websocket(path)` — websocket route
- → `await ws.accept()` — complete the handshake
- → `receive_text()` — waits for the next message
- → `send_json()` — send a dict as JSON
- → `WebSocketDisconnect` — client left

OTHER FRAMEWORK

- Flask — no native support; Flask-SocketIO or Flask-Sock
- Django — Django Channels with consumers

PREFER / AVOID

- Avoid an in-memory manager with many workers · Prefer Redis pub/sub broadcast
- Avoid unauthenticated sockets · Prefer a token in query or first message

WEBSOCKET API

METHOD	DOES
`accept()`	open
`receive_text / bytes / json`	read
`send_text / bytes / json`	write
`close(code=1000)`	close
`iter_text()`	async for loop

CLOSE CODES

CODE	MEANING
`1000`	normal
`1001`	going away
`1008`	policy violation (auth)
`1011`	server error

⚠ COMMON MISTAKES

- Forgetting `accept()` : the client sees an immediate close
- Raising `HTTPException` in a websocket; use `WebSocketException` or `close()`
- Broadcasting while iterating a list that a disconnect modifies

✓ TIPS & BEST PRACTICES

- Send heartbeats (ping) to keep proxies from closing idle sockets
- Configure Nginx with `Upgrade` / `Connection` headers (sheet 45)
- Prefer SSE (sheet 39) when data only flows server → client

QUICK SUMMARY

- `@app.websocket` — real-time endpoint
- `accept` → loop `receive` / `send`
- `WebSocketDisconnect` — clean up
- multi-worker — Redis to broadcast

Streaming & Server-Sent Events

StreamingResponse · async generators · text/event-stream · sse-starlette · LLM streaming · large downloads

DESCRIPTION

`StreamingResponse` sends a body in chunks from a generator, so the client starts receiving before you finish. Use it for large exports and streaming AI output. **Server-Sent Events** (SSE) are a simple text format over a stream (`data: ...\n\n`) that browsers read with `EventSource` and auto-reconnect.

NOTES

- Generators can be sync or async
- SSE is one-way (server → client), plain HTTP
- Disable proxy buffering (`X-Accel-Buffering: no`)
- `sse-starlette` adds `EventSourceResponse` with pings

CSV EXPORT + SSE

```python
import asyncio, csv, io
from fastapi.responses import StreamingResponse

@app.get("/export.csv")
def export(db: DbSession):
    def rows():
        buf = io.StringIO(); w = csv.writer(buf)
        w.writerow(["id", "title"])
        for t in db.scalars(select(Todo)).yield_per(500):
            w.writerow([t.id, t.title])
            yield buf.getvalue(); buf.seek(0); buf.truncate()
        yield buf.getvalue()
    return StreamingResponse(rows(), media_type="text/csv",
        headers={"Content-Disposition": "attachment"})

@app.get("/events")
async def events(request: Request):
    async def gen():
        n = 0
        while not await request.is_disconnected():
            n += 1
            yield f"event: tick\ndata: {n}\n\n"
            await asyncio.sleep(1)
    return StreamingResponse(gen(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache"})
```

CODE EXPLAINED

- → `rows()` — yields CSV chunks as rows are read
- → `yield_per(500)` — stream rows from the DB
- → `Content-Disposition` — download as a file
- → SSE frame — `event:` + `data:` + blank line
- → `is_disconnected()` — stop when the client leaves

OTHER FRAMEWORK

- Flask — `Response(stream_with_context(gen()), mimetype=...)`
- Django — `StreamingHttpResponse(gen())`

PREFER / AVOID

- Avoid building a huge string in memory · Prefer a generator
- Avoid polling every second · Prefer SSE for server pushes

SSE FIELDS

FIELD	MEANING
`data:`	payload (repeat for lines)
`event:`	event name
`id:`	last-event ID for resume
`retry:`	reconnect delay ms
`: comment`	keep-alive ping

STREAM VS WEBSOCKET

NEED	USE
`server → client only`	SSE
`two-way, low latency`	WebSocket
`big file`	FileResponse / stream
`LLM tokens`	SSE or chunked stream

⚠ COMMON MISTAKES

- Proxy buffering hides streaming: client gets everything at the end
- Using the request session after the route returned in a stream (it may be closed)
- Forgetting the blank line that ends each SSE event

✓ TIPS & BEST PRACTICES

- Read the stream with `EventSource` in browsers or `httpx.stream()` in Python
- Send `id:` so clients resume with `Last-Event-ID`
- Set `Cache-Control: no-cache` on SSE

QUICK SUMMARY

- `StreamingResponse` — chunked body from a generator
- SSE — `text/event-stream`, one-way
- `is_disconnected()` — stop early
- turn off proxy buffering

TestClient Basics

DESCRIPTION

TestClient calls your app in-process with an `httpx`-style API, so tests are fast and need no running server. Write normal **pytest** functions that make requests and assert on `status_code` and `json()`. Use it as a context manager (`with TestClient(app)`) when you need lifespan startup and shutdown to run.

NOTES

- Test functions can be plain `def`, even for async routes
- `TestClient` needs `httpx` (in `fastapi[standard]`)
- Run with `pytest` from the project root
- See the pytest set for fixtures and parametrize

TESTS/TEST_TODOS.PY

```python
import pytest
from fastapi.testclient import TestClient
from app.main import app

@pytest.fixture
def client():
    with TestClient(app) as c:              # runs lifespan
        yield c

def test_create_and_get(client):
    r = client.post("/todos", json={"title": "Buy milk"})
    assert r.status_code == 201
    todo = r.json()
    assert todo["title"] == "Buy milk"

    r = client.get(f"/todos/{todo['id']}")
    assert r.status_code == 200
    assert r.json() == todo

def test_validation(client):
    r = client.post("/todos", json={})
    assert r.status_code == 422
    assert r.json()["detail"][0]["loc"] == ["body", "title"]
```

CODE EXPLAINED

- → `with TestClient(app)` — starts and stops lifespan
- → fixture — one client per test
- → `post(json=...)` — sends JSON body
- → `r.json()` — parsed response
- → 422 test — pins the error shape

OTHER FRAMEWORK

- Flask — `app.test_client()` (sheet 57)
- Django — `self.client.get()` in `TestCase`

PREFER / AVOID

- Avoid starting a real server for tests · Prefer `TestClient`
- Avoid one giant test · Prefer one behavior per test

REQUEST OPTIONS

ARG	SENDS
`json={...}`	JSON body
`data={...}`	form fields
`files={"file": (...)}`	upload
`params={...}`	query string
`headers={...}`	headers
`cookies={...}`	cookies

RESPONSE CHECKS

ATTRIBUTE	VALUE
`status_code`	int
`json()`	parsed body
`text / content`	str / bytes
`headers["x"]`	header
`cookies`	set cookies

⚠ COMMON MISTAKES

- Plain `TestClient(app)` with lifespan-created resources: startup never ran
- Tests sharing a real database and depending on order
- Asserting on whole error messages that change between versions

✓ TIPS & BEST PRACTICES

- Put the `client` fixture in `tests/conftest.py`
- Use `pytest -x -q` for quick feedback
- Use `client.headers.update(...)` for auth on every request

QUICK SUMMARY

- `TestClient(app)` — in-process requests
- `with` — run lifespan
- assert `status_code` + `json()`
- pytest fixtures — reusable setup

Overrides & Async Tests

DESCRIPTION

`app.dependency_overrides` swaps real dependencies (DB session, current user, settings) for test versions without touching app code. When tests themselves must be async (to call async helpers or share an event loop with an async DB), use **`httpx.AsyncClient`** with **`ASGITransport`** and mark tests with `anyio`.

NOTES

- Override the exact callable used in `Depends()`
- A SQLite file or in-memory DB per test run keeps tests isolated
- `ASGITransport` does not run lifespan; use `asgi-lifespan` if needed
- `anyio` comes with FastAPI; `pytest-asyncio` also works

CONFTEST.PY

```python
import pytest
from httpx import ASGITransport, AsyncClient
from sqlmodel import Session, SQLModel, create_engine
from sqlmodel.pool import StaticPool
@pytest.fixture
def session():
    engine = create_engine("sqlite://", poolclass=StaticPool,
                connect_args={"check_same_thread": False})
    SQLModel.metadata.create_all(engine)
    with Session(engine) as s: yield s

@pytest.fixture
def client(session):
    app.dependency_overrides[get_db] = lambda: session
    app.dependency_overrides[get_current_user] = lambda: User(id=1)
    with TestClient(app) as c:
        yield c
    app.dependency_overrides.clear()

@pytest.mark.anyio
async def test_async():
    t = ASGITransport(app=app)
    async with AsyncClient(transport=t, base_url="http://t") as ac:
        r = await ac.get("/health")
    assert r.status_code == 200
```

CODE EXPLAINED

- → `sqlite://` + `StaticPool` — one shared in-memory DB
- → `dependency_overrides[get_db]` — tests use that session
- → fake `get_current_user` — skip real auth
- → `ASGITransport` — async client against the app

OTHER FRAMEWORK

- Flask — `app.config["TESTING"] = True` + a test DB URI
- Django — test DB created automatically per run

PREFER / AVOID

- Avoid patching modules with `mock.patch` · Prefer `dependency_overrides`
- Avoid async tests unless needed · Prefer `TestClient` + sync tests

OVERRIDE TARGETS

DEPENDENCY	TEST VERSION
`get_db`	test session
`get_current_user`	fixed user
`get_settings`	test Settings
`get_http_client`	mock client (respx)
`get_mailer`	outbox list

ASYNC TEST SETUP

NEED	TOOL
`async test functions`	@pytest.mark.anyio
`client`	AsyncClient + ASGITransport
`lifespan`	asgi-lifespan LifespanManager
`mock HTTP`	respx

⚠ COMMON MISTAKES

- Overriding a function with a different signature than the one in `Depends()`
- In-memory SQLite without `StaticPool`: each connection gets an empty DB
- Mixing `pytest-asyncio` and `anyio` markers in one suite

✓ TIPS & BEST PRACTICES

- Add `anyio_backend` fixture returning `"asyncio"` to skip trio runs
- Use factories (`make_user()`) for test data
- Run tests in CI against the real DB engine too

QUICK SUMMARY

- `dependency_overrides` — swap dependencies
- in-memory DB + `StaticPool` — isolated tests
- `AsyncClient` + `ASGITransport` — async tests
- clear overrides after each test

Static Files & Templates

DESCRIPTION

FastAPI can serve HTML too. **`StaticFiles`** serves CSS, JS and images from a folder, mounted at a path. **`Jinja2Templates`** renders HTML templates with your data. Pair them with **HTMX** for interactive pages without a JavaScript framework.

NOTES

- `jinja2` is included in `fastapi[standard]`
- Mounted apps are independent: no docs, no route deps
- `TemplateResponse` takes `request` as its first argument
- Use `response_class=HTMLResponse` for clean docs

MAIN.PY

```python
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse

app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

@app.get("/todos/page", response_class=HTMLResponse)
def todo_page(request: Request, db: DbSession):
    todos = list_todos(db)
    return templates.TemplateResponse(request, "todos.html",
                                      {"todos": todos})
```

CODE EXPLAINED

- → `app.mount(path, StaticFiles(...), name=)` — static folder
- → `Jinja2Templates(directory=)` — template loader
- → `TemplateResponse(request, name, context)` — render
- → `url_for('static', path=)` — static file URL
- → `{% for %}...{% else %}` — empty-list fallback

TEMPLATES/TODOS.HTML

```jinja
<link rel="stylesheet" href="{{ url_for('static', path='app.css') }}">
<ul>
  {% for t in todos %}
    <li class="{{ 'done' if t.done }}">{{ t.title }}</li>
  {% else %}
    <li>Nothing to do!</li>
  {% endfor %}
</ul>
```

OTHER FRAMEWORK

- Flask — `render_template()` + `static/` folder by default (sheet 49)
- Django — `render()` + `{% static %}` tag

PREFER / AVOID

- Avoid building HTML strings in Python · Prefer templates (auto-escaped)
- Avoid serving static files via Python in prod · Prefer Nginx or a CDN

JINJA2 SYNTAX

SYNTAX	DOES
`{{ value }}`	output, escaped
`{% if %}...{% endif %}`	condition
`{% for x in xs %}`	loop
`{% extends "base.html" %}`	inherit
`{% block body %}`	override area
`{{ x | upper }}`	filter

STATICFILES OPTIONS

OPTION	EFFECT
`directory=`	folder to serve
`html=True`	serve index.html
`check_dir=False`	allow missing dir
`follow_symlink`	allow symlinks

⚠ COMMON MISTAKES

- Old `TemplateResponse("x.html", {"request": request})` signature is deprecated
- Mounting static at `/` and hiding all API routes
- Using `|safe` on user input: XSS

✓ TIPS & BEST PRACTICES

- Return template partials for HTMX requests (`HX-Request` header)
- Use `html=True` to serve a built SPA folder
- Cache static files with long `max-age` + hashed names

QUICK SUMMARY

- `StaticFiles` + `mount()` — static assets
- `Jinja2Templates` — server-rendered HTML
- `TemplateResponse(request, ...)` — render
- HTMX — interactivity without a JS framework

OpenAPI Customization & Client Generation

DESCRIPTION

The OpenAPI schema is a product of its own: clients are generated from it, docs are read from it. Add **tag descriptions**, hide internal routes, give operations **stable IDs** for nice generated method names, and patch the schema for anything FastAPI doesn't model. Then generate a typed TypeScript or Python client.

NOTES

- Schema is built once and cached on `app.openapi_schema`
- `openapi_url=None` turns off schema and both doc UIs
- Operation IDs default to `function_name_path_method`
- OpenAPI 3.1 is the version FastAPI emits

TAGS, IDS, CUSTOM SCHEMA

```python
from fastapi.openapi.utils import get_openapi
from fastapi.routing import APIRoute

def route_id(route: APIRoute) -> str:
    return f"{route.tags[0]}-{route.name}"      # todos-list_todos

app = FastAPI(
    openapi_tags=[
        {"name": "todos", "description": "Manage todo items"},
        {"name": "auth", "description": "Login and tokens"}],
    generate_unique_id_function=route_id,
)

@app.get("/internal/metrics", include_in_schema=False)
def metrics(): ...

def custom_openapi():
    if app.openapi_schema:
        return app.openapi_schema
    schema = get_openapi(title="Todo API", version="2.0.0",
                         routes=app.routes)
    schema["info"]["x-logo"] = {"url": "/static/logo.png"}
    app.openapi_schema = schema
    return schema

app.openapi = custom_openapi
```

CODE EXPLAINED

- → `openapi_tags` — descriptions and order of tags
- → `generate_unique_id_function` — clean operation IDs
- → `include_in_schema=False` — hide a route
- → `get_openapi()` — build the default schema
- → `app.openapi = ...` — replace the generator

OTHER FRAMEWORK

- Flask — APIFlask / flask-smorest generate OpenAPI from schemas
- Django — `drf-spectacular` with `@extend_schema`

PREFER / AVOID

- Avoid hand-written API clients · Prefer generated ones from `/openapi.json`
- Avoid unstable operation IDs · Prefer a custom ID function

CLIENT GENERATORS

TOOL	OUTPUT
`@hey-api/openapi-ts`	TypeScript + fetch/axios
`openapi-typescript`	TS types only
`orval`	TS + React Query hooks
`openapi-python-client`	typed Python client
`openapi-generator`	many languages

SCHEMA SWITCHES

SETTING	EFFECT
`openapi_url=None`	no schema, no docs
`docs_url=None`	no Swagger UI
`redoc_url=None`	no ReDoc
`swagger_ui_parameters={...}`	UI options
`servers=[{"url": ...}]`	base URLs in docs

⚠ COMMON MISTAKES

- Changing route function names and breaking generated client method names
- Forgetting to cache the custom schema: rebuilt on every `/openapi.json` hit
- Hiding routes from the schema and assuming that secures them

✓ TIPS & BEST PRACTICES

- Regenerate clients in CI and fail on uncommitted diffs
- Use `swagger_ui_parameters={"persistAuthorization": True}`
- Export the schema with `app.openapi()` in a script for frontends

QUICK SUMMARY

- `openapi_tags` — documented tag groups
- unique ID function — clean client names
- custom `openapi()` — patch the schema
- generate clients from `/openapi.json`

Performance & Production Config

ORJSONResponse · response caching · connection pooling · root_path · docs off in prod · logging

DESCRIPTION

FastAPI is fast out of the box; most real gains come from **fewer and cheaper I/O calls**. Reuse connection pools and HTTP clients, cache hot responses, pick a faster JSON encoder, and paginate. Production config also means correct **proxy settings** (`root_path`, forwarded headers), structured logging, and no debug-only features.

NOTES

- Measure first: `py-spy`, OpenTelemetry, or simple timing middleware
- Pydantic v2 serialization is already Rust-fast
- `orjson` helps with large JSON responses
- Uvicorn `--proxy-headers` trusts `X-Forwarded-*`

PRODUCTION SETTINGS

```
import logging
from fastapi import FastAPI
from fastapi.responses import ORJSONResponse

logging.basicConfig(level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s %(message)s")
log = logging.getLogger("todo")

app = FastAPI(
    default_response_class=ORJSONResponse,    # faster JSON
    root_path="/api",                         # behind proxy at /api
    docs_url=None if settings.prod else "/docs",
    redoc_url=None,
)

@app.get("/todos/stats")
async def stats(response: Response, cache: Redis):
    if hit := await cache.get("stats"):
        return ORJSONResponse(content=json.loads(hit))
    data = await compute_stats()
    await cache.set("stats", json.dumps(data), ex=60)
    response.headers["Cache-Control"] = "public, max-age=60"
    return data
```

CODE EXPLAINED

- → `default_response_class=ORJSONResponse` — app-wide fast JSON
- → `root_path` — correct URLs and docs behind a path prefix
- → docs disabled in prod via settings
- → Redis cache — 60 s for an expensive query
- → `Cache-Control` — let CDNs and browsers cache

OTHER FRAMEWORK

- Flask — `flask-caching`; `ProxyFix` for proxy headers
- Django — cache framework + `@cache_page`

PREFER / AVOID

- Avoid N+1 queries · Prefer eager loading / joins
- Avoid `print()` debugging in prod · Prefer `logging` with levels

QUICK WINS

CHANGE	WHY
reuse httpx client	connection pooling
DB pool sizing	avoid connection waits
paginate everything	smaller responses
cache hot reads	skip the DB
GZip middleware	less bandwidth
uvloop + httptools	faster server loop

PRODUCTION CHECKLIST

ITEM	SETTING
no reload	fastapi run
workers	--workers N
proxy headers	--proxy-headers
allowed hosts	TrustedHostMiddleware
HTTPS	terminate at proxy
health check	/health

⚠ COMMON MISTAKES

- Optimizing JSON speed while making 50 DB queries per request
- Missing `--forwarded-allow-ips`, so client IPs all show as the proxy
- Caching per-user data under a shared key

✓ TIPS & BEST PRACTICES

- Add OpenTelemetry instrumentation for traces across services
- Set `pool_size` + `max_overflow` per worker, not per server
- Load-test with realistic data sizes

QUICK SUMMARY

- measure, then optimize I/O first
- `ORJSONResponse` — faster JSON
- `root_path` + proxy headers — behind a proxy
- logging + no docs/reload — prod config

Deployment

DESCRIPTION

In production, run FastAPI with **`fastapi run`** (or Uvicorn directly) with several **worker processes**, inside a container, behind a **reverse proxy** that handles HTTPS. Use one worker per container when an orchestrator like Kubernetes scales containers for you; use several workers on a single VM.

NOTES

- Workers ≈ CPU cores for one VM/container
- In Kubernetes: 1 worker per pod, scale pods
- Proxy terminates TLS; app speaks plain HTTP inside
- Run migrations as a separate deploy step

DOCKERFILE

```dockerfile
FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY ./app ./app
EXPOSE 8000
CMD ["fastapi", "run", "app/main.py", "--port", "8000", \
     "--workers", "4", "--proxy-headers"]
```

CODE EXPLAINED

- → slim base image — smaller, faster pulls
- → copy requirements first — cached dependency layer
- → exec-form `CMD` — clean signals and shutdown
- → `--proxy-headers` — trust `X-Forwarded-*`
- → `Upgrade` headers — needed for websockets

NGINX REVERSE PROXY

```nginx
server {
    listen 443 ssl;
    server_name api.example.com;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;     # websockets
        proxy_set_header Connection "upgrade";
    }
}
```

OTHER FRAMEWORK

- Flask — Gunicorn or Waitress behind Nginx (sheet 57)
- Django — Gunicorn (WSGI) or Uvicorn (ASGI) + `collectstatic`

PREFER / AVOID

- Avoid `fastapi dev` / `--reload` in prod · Prefer `fastapi run`
- Avoid running as root in the container · Prefer a non-root `USER`

SERVER OPTIONS

COMMAND	WHEN
`fastapi run --workers 4`	simple, single host
`uvicorn app.main:app --workers 4`	same, direct
`gunicorn -k uvicorn_worker.UvicornWorker`	process manager features
`1 worker per pod`	Kubernetes / ECS

WHERE TO HOST

OPTION	NOTES
`VPS + Docker + Nginx`	full control
`Fly.io / Render / Railway`	push to deploy
`AWS ECS / Cloud Run`	containers, autoscale
`Kubernetes`	big fleets
`FastAPI Cloud`	official managed host

⚠ COMMON MISTAKES

- Many workers per pod and autoscaling pods: double counting, memory blowup
- Forgetting forwarded headers: wrong scheme in redirects and docs
- Baking secrets into the image instead of env vars

✓ TIPS & BEST PRACTICES

- Add a `HEALTHCHECK` or orchestrator probe on `/health`
- Pin the Python base image tag and rebuild for security updates
- See the Docker and Ubuntu sets for container and server setup

QUICK SUMMARY

- `fastapi run --workers N` — prod server
- Docker — repeatable builds
- Nginx / proxy — TLS, headers, websockets
- 1 worker per pod — when the platform scales

Flask Introduction & First App

DESCRIPTION

Flask is a small, mature WSGI framework from the Pallets project. It gives you routing, a request object, templates (**Jinja2**) and a dev server, and leaves the database, forms and auth to **extensions** you pick. That makes it quick to learn and flexible, at the cost of making more choices yourself than in Django or FastAPI.

NOTES

- Released 2010 by Armin Ronacher; current major is Flask 3.x
- Built on Werkzeug (WSGI toolkit) and Jinja2
- Return a dict or list from a view → JSON
- `--debug` enables reload and the interactive debugger

UNDER THE HOOD

- Werkzeug parses the WSGI `environ` into a `Request` object
- Flask pushes app + request contexts so `request` works as a global (sheet 52)
- The dev server is Werkzeug's: never use it in production

APP.PY

```python
from flask import Flask, request

app = Flask(__name__)                  # __name__ locates templates/static

@app.get("/")
def index():
    return "<h1>Hello, Flask</h1>"     # str → HTML

@app.get("/todos/<int:todo_id>")
def get_todo(todo_id: int):
    done = request.args.get("done", default=False, type=bool)
    return {"id": todo_id, "done": done}   # dict → JSON

@app.post("/todos")
def create_todo():
    data = request.get_json()
    return {"title": data["title"]}, 201
```

RUN IT

```bash
pip install flask
flask --app app run --debug          # http://127.0.0.1:5000
```

CODE EXPLAINED

- → `Flask(__name__)` — app object; finds `templates/` and `static/`
- → `@app.get` — route shortcut (Flask 2.0+)
- → `request.args.get(..., type=)` — query param
- → return `(body, 201)` — body plus status

OTHER FRAMEWORK

- FastAPI — typed params replace `request.args` / `get_json()` (sheet 01)
- Django — project + app, URLconf, views

PREFER / AVOID

- Avoid `app.run()` in production · Prefer Gunicorn (sheet 57)
- Avoid `FLASK_ENV` (removed) · Prefer `--debug` or `FLASK_DEBUG=1`

FLASK BUILDING BLOCKS

PIECE	PROVIDES
Werkzeug	WSGI, routing, request
Jinja2	templates
Click	flask CLI
ItsDangerous	signed cookies
MarkupSafe	HTML escaping

POPULAR EXTENSIONS

EXTENSION	ADDS
Flask-SQLAlchemy	ORM (54)
Flask-Migrate	migrations (54)
Flask-WTF	forms + CSRF (50)
Flask-Login	sessions (55)
Flask-CORS	CORS
Flask-Caching	caching

⚠ COMMON MISTAKES

- Naming the file `flask.py`: shadows the package
- Leaving `--debug` on a public server: the debugger allows code execution
- Using `request` outside a request: "Working outside of request context"

✓ TIPS & BEST PRACTICES

- Use `@app.get` / `@app.post` shortcuts for clarity
- Put settings in a `.flaskenv` file (needs `python-dotenv`)
- Move to the app factory early (sheet 51)

QUICK SUMMARY

- `Flask(__name__)` — WSGI app
- `@app.get(path)` — routes
- dict return — JSON
- `flask --app app run --debug` — dev server

Routing & URL Building

@app.route · @app.get / @app.post · <int:id> converters · methods=[…] · url_for() · redirect() · trailing slashes

DESCRIPTION

Flask maps URLs to view functions with `@app.route` (or method shortcuts). **Converters** like `<int:id>` parse path parts. Build URLs with **`url_for()`**, which uses the function name (the endpoint), so changing a path never breaks links. Trailing slashes matter: they decide redirect vs 404.

NOTES

- Default methods are GET (plus HEAD, OPTIONS)
- Endpoint name defaults to the function name
- `/todos/` with slash: `/todos` redirects to it
- `/todos` without slash: `/todos/` is a 404

ROUTES & URLS

```python
from flask import Flask, redirect, url_for, abort

app = Flask(__name__)

@app.route("/todos", methods=["GET", "POST"])
def todos():
    ...

@app.get("/todos/<int:todo_id>")
def show_todo(todo_id):
    ...

@app.get("/files/<path:name>")
def files(name): ...                          # "a/b/c.txt"

@app.get("/go/<slug>")
def go(slug):
    return redirect(url_for("show_todo", todo_id=7))    # → /todos/7

with app.test_request_context():
    url_for("todos")                          # '/todos'
    url_for("todos", page=2)                  # '/todos?page=2'
    url_for("static", filename="app.css")     # '/static/app.css'
    url_for("show_todo", todo_id=3, _external=True)
```

CODE EXPLAINED

→ `methods=[...]` — allow several methods on one view

→ `<int:todo_id>` — converted to `int`, else 404

→ `url_for("show_todo", todo_id=7)` — build by endpoint

→ extra kwargs → query string

→ `_external=True` — absolute URL

OTHER FRAMEWORK

- FastAPI — `{todo_id}` + type hint; `request.url_for("name", ...)`
- Django — `<int:pk>` + `reverse("name", args=[...])`

PREFER / AVOID

- Avoid hard-coded URLs in templates · Prefer `url_for()`
- Avoid one view with `if request.method ==` branches · Prefer `@app.get` / `@app.post`

CONVERTERS

CONVERTER	MATCHES
`<name>` / `<string:name>`	text without /
`<int:id>`	positive int
`<float:x>`	float
`<path:p>`	text with /
`<uuid:id>`	UUID

RESPONSE HELPERS

HELPER	DOES
`redirect(url, 302)`	redirect
`url_for(endpoint, **kw)`	build URL
`abort(404)`	error response
`make_response(...)`	build Response
`send_file(path)`	file download

⚠ COMMON MISTAKES

- Two views with the same function name: "View function mapping is overwriting" error
- Inconsistent trailing slashes causing surprise 404s
- Calling `url_for` outside a request or app context

✓ TIPS & BEST PRACTICES

- `flask --app app routes` lists every route and endpoint
- Use `endpoint=` to name views explicitly when needed
- Blueprint endpoints are prefixed: `url_for("todos.show")`

QUICK SUMMARY

- `@app.route` / `@app.get` — map URLs
- converters — typed path parts
- `url_for(endpoint)` — never hard-code URLs
- trailing slash — redirect vs 404

Request & Response

DESCRIPTION

Flask's **request** object holds everything the client sent: query string, form data, JSON, files, headers and cookies. A view can return a string, a dict or list (JSON), a Response, or a **tuple** of body, status and headers. Use **make_response()** when you need to set cookies or tweak headers before returning.

NOTES

- `request` is a context-local; import it, don't pass it
- `args` / `form` are `MultiDict`s: `.get()`, `.getlist()`
- `get_json()` returns `None` or 415 for non-JSON
- `jsonify` is optional now; dicts become JSON

READING & RESPONDING

```python
from flask import request, jsonify, make_response

@app.post("/todos")
def create_todo():
    data = request.get_json(silent=True) or {}
    title = data.get("title", "").strip()
    if not title:
        return {"error": "title is required"}, 400
    todo = repo.add(title)
    return todo, 201, {"Location": f"/todos/{todo['id']}"}

@app.get("/search")
def search():
    q = request.args.get("q", "")
    page = request.args.get("page", 1, type=int)
    tags = request.args.getlist("tag")       # ?tag=a&tag=b
    agent = request.headers.get("User-Agent")
    return jsonify(results=find(q, page, tags))

@app.post("/prefs")
def prefs():
    resp = make_response({"ok": True})
    resp.set_cookie("theme", request.form["theme"], httponly=True)
    return resp
```

CODE EXPLAINED

- → `get_json(silent=True)` — `None` instead of an error
- → return `(body, status, headers)` tuple
- → `args.get(..., type=int)` — convert, default on failure
- → `getlist("tag")` — repeated params
- → `make_response()` — then set cookies/headers

OTHER FRAMEWORK

- FastAPI — typed params + Pydantic replace manual parsing (sheets 07–08)
- Django — `request.GET`, `request.POST`, `json.loads(request.body)`

PREFER / AVOID

- Avoid `request.form["x"]` when it may be missing (400) · Prefer `.get()`
- Avoid hand-validating big payloads · Prefer Pydantic or Marshmallow

REQUEST ATTRIBUTES

ATTRIBUTE	HOLDS
`args`	query string
`form`	form fields
`get_json()`	JSON body
`files`	uploads
`headers / cookies`	metadata
`method / path / url`	request line

RETURN VALUES

RETURN	BECOMES
`"text"`	HTML 200
`{...} or [...]`	JSON 200
`(body, 201)`	with status
`(body, 201, {h: v})`	with headers
`Response(...)`	as-is

⚠ COMMON MISTAKES

- `request.form["x"]` on a missing key returns a 400 page, confusing API clients
- Calling `get_json()` without `Content-Type: application/json`
- Reading `request.data` for form posts: it's empty after parsing

✓ TIPS & BEST PRACTICES

- Validate JSON with Pydantic: `Todo.model_validate(request.get_json())`
- Set `MAX_CONTENT_LENGTH` to cap upload size
- Use `request.get_json(force=True)` only for clients you control

QUICK SUMMARY

- `request.args` / `form` / `get_json()` — inputs
- dict return — JSON
- tuple — body, status, headers
- `make_response()` — cookies and headers

Templates with Jinja2

DESCRIPTION

`render_template()` renders a Jinja2 file from the `templates/` folder with the variables you pass. Templates use **inheritance**: a `base.html` defines blocks that child templates fill. Output is **auto-escaped** in `.html` templates, and `url_for`, `request`, `session` and `g` are available inside them.

NOTES

- Templates live in `templates/`, assets in `static/`
- `{{ }}` outputs; `{% %}` runs logic; `{# #}` comments
- Auto-escaping is on for `.html`, `.htm`, `.xml`
- Context processors inject variables into every template

VIEW

```python
from flask import render_template

@app.get("/todos")
def todo_list():
    return render_template("todos.html", todos=repo.all(),
                           title="My Todos")
```

CODE EXPLAINED

- → `render_template(name, **ctx)` — render with variables
- → `{% extends %}` + `{% block %}` — layout inheritance
- → `url_for('static', filename=)` — asset URL
- → `loop.index` — 1-based loop counter
- → `| title` — filter: Title Case

TEMPLATES/BASE.HTML + TODOS.HTML

```jinja
<!-- base.html -->
<title>{% block title %}App{% endblock %}</title>
<link rel="stylesheet"
      href="{{ url_for('static', filename='app.css') }}">
<main>{% block content %}{% endblock %}</main>

<!-- todos.html -->
{% extends "base.html" %}
{% block title %}{{ title }}{% endblock %}
{% block content %}
  <ul>
  {% for t in todos %}
    <li>{{ loop.index }}. {{ t.title | title }}
        {% if t.done %}✔{% endif %}</li>
  {% else %}
    <li>Nothing to do!</li>
  {% endfor %}
  </ul>
{% endblock %}
```

OTHER FRAMEWORK

- FastAPI — `Jinja2Templates` + `TemplateResponse` (sheet 42)
- Django — Django template language: similar tags, fewer expressions

PREFER / AVOID

- Avoid `| safe` on user content · Prefer auto-escaping
- Avoid repeating layout HTML · Prefer `base.html` + blocks

COMMON FILTERS

FILTER	RESULT
`upper / lower / title`	case
`default("-")`	fallback
`length`	count
`join(", ")`	join list
`truncate(40)`	shorten
`safe`	skip escaping

TEMPLATE TAGS

TAG	DOES
`{% include "x.html" %}`	insert partial
`{% macro f() %}`	reusable snippet
`{% set x = 1 %}`	variable
`{% with %}`	scoped vars
`{{ super() }}`	parent block content

⚠ COMMON MISTAKES

- Putting templates in the wrong folder (e.g. next to `app.py`, not in `templates/`)
- Using `| safe` to fix double escaping instead of fixing the data
- Heavy logic in templates; compute in the view

✓ TIPS & BEST PRACTICES

- Use `@app.context_processor` for values every page needs
- Use macros for repeated form fields
- `flask shell` + `render_template_string` for quick experiments

QUICK SUMMARY

- `render_template()` — Jinja2 from `templates/`
- `extends` / `block` — layouts
- filters — `{{ x | upper }}`
- auto-escape — XSS protection by default

Forms & Flask-WTF

DESCRIPTION

Flask-WTF integrates WTForms with Flask: define a form class with fields and validators, render it in a template, and call **`validate_on_submit()`** in the view. It adds **CSRF protection** automatically. Use **`flash()`** to queue one-time messages shown on the next page.

NOTES

- `pip install flask-wtf` (needs `SECRET_KEY`)
- `validate_on_submit()` = POST and all validators pass
- `form.hidden_tag()` renders the CSRF token
- Email validation needs `email-validator`

FORM + VIEW

```
from flask import flash, redirect, render_template, url_for
from flask_wtf import FlaskForm
from wtforms import StringField, IntegerField, SubmitField
from wtforms.validators import (DataRequired, Length,
                                NumberRange, ValidationError)

class TodoForm(FlaskForm):
    title = StringField("Title",
                        validators=[DataRequired(), Length(max=100)])
    priority = IntegerField("Priority", default=3,
                            validators=[NumberRange(1, 5)])
    submit = SubmitField("Add")

    def validate_title(self, field):          # custom rule
        if repo.exists(field.data):
            raise ValidationError("Already on the list")

@app.route("/todos/new", methods=["GET", "POST"])
def new_todo():
    form = TodoForm()
    if form.validate_on_submit():
        repo.add(form.title.data, form.priority.data)
        flash("Todo added", "success")
        return redirect(url_for("todo_list"))
    return render_template("new.html", form=form)
```

CODE EXPLAINED

- → `FlaskForm` — WTForms + CSRF
- → `validators=[...]` — rules per field
- → `validate_<field>()` — custom check
- → `validate_on_submit()` — POST + valid
- → `flash()` + redirect — message on the next page

OTHER FRAMEWORK

- FastAPI — `Form()` params + Pydantic validation (sheet 10)
- Django — `forms.Form` / `ModelForm` with `{% csrf_token %}`

PREFER / AVOID

- Avoid rendering the page directly after a successful POST · Prefer redirect
- Avoid disabling CSRF globally · Prefer `@csrf.exempt` on specific API views

FIELD TYPES

FIELD	HTML
`StringField`	text
`PasswordField`	password
`TextAreaField`	textarea
`IntegerField / DecimalField`	number
`SelectField`	select
`BooleanField`	checkbox

VALIDATORS

VALIDATOR	CHECKS
`DataRequired()`	not empty
`Length(min, max)`	string length
`Email()`	email format
`NumberRange(a, b)`	range
`EqualTo("password")`	fields match
`Optional()`	allow empty

⚠ COMMON MISTAKES

- Missing `SECRET_KEY`: CSRF fails with a cryptic error
- Forgetting `{{ form.hidden_tag() }}` in the template: every POST fails validation
- Reading `request.form` instead of `form.field.data` and skipping validation

✓ TIPS & BEST PRACTICES

- Render errors with `{% for e in form.title.errors %}`
- Show flashes in `base.html` with `get_flashed_messages()`
- Use `CSRFProtect(app)` to protect non-WTForms POSTs too

QUICK SUMMARY

- `FlaskForm` — fields + validators + CSRF
- `validate_on_submit()` — POST and valid
- `flash()` — one-time messages
- POST → redirect → GET

Blueprints & the App Factory

DESCRIPTION

A **Blueprint** groups related routes, templates and static files, like FastAPI's `APIRouter`. The **application factory** (`create_app()`) builds and configures the app inside a function, which makes testing with different configs easy and avoids circular imports. Extensions are created once and attached with `init_app()`.

NOTES

- Endpoints become `blueprint.view` (e.g. `todos.show`)
- `flask --app myapp run` finds `create_app()` automatically
- `instance/` holds local config and the SQLite DB, not committed
- Blueprints can have their own `templates/` folders

MYAPP/__INIT__.PY + TODOS.PY

```python
# myapp/todos.py
from flask import Blueprint, render_template
bp = Blueprint("todos", __name__, url_prefix="/todos")

@bp.get("/")
def index():
    return render_template("todos/index.html")

# myapp/__init__.py
from flask import Flask

def create_app(config: dict | None = None) -> Flask:
    app = Flask(__name__, instance_relative_config=True)
    app.config.from_object("myapp.config.Default")
    app.config.update(config or {})
    from .extensions import db      # db = SQLAlchemy()
    db.init_app(app)
    from . import todos, auth
    app.register_blueprint(todos.bp)
    app.register_blueprint(auth.bp)
    return app
```

CODE EXPLAINED

- → `Blueprint(name, __name__, url_prefix=)` — route group
- → `create_app(config)` — build a configured app
- → `db.init_app(app)` — bind extension to this app
- → imports inside the factory — no circular imports

OTHER FRAMEWORK

- FastAPI — `APIRouter` + `include_router()` (sheet 25)
- Django — apps + `include()`; settings modules per environment

PREFER / AVOID

- Avoid a module-level `app = Flask()` imported everywhere · Prefer `create_app()`
- Avoid `url_for("show")` across blueprints · Prefer `url_for("todos.show")`

PACKAGE LAYOUT

PATH	HOLDS
myapp/__init__.py	create_app()
myapp/extensions.py	db, login, csrf
myapp/todos.py	todos blueprint
myapp/templates/	Jinja2 files
myapp/static/	assets
instance/config.py	local secrets

BLUEPRINT OPTIONS

OPTION	EFFECT
url_prefix=	path prefix
template_folder=	own templates
static_folder=	own static files
subdomain=	subdomain routing
@bp.before_request	hook for this bp

⚠ COMMON MISTAKES

- Creating extensions inside `create_app()`: other modules can't import them
- Relative `url_for(".show")` used outside its blueprint
- Committing `instance/` with secrets

✓ TIPS & BEST PRACTICES

- Pass `{"TESTING": True, ...}` to `create_app()` in tests
- Register error handlers with `@bp.app_errorhandler` for app-wide pages
- Keep one blueprint per feature, like routers

QUICK SUMMARY

- Blueprint — group of routes
- `create_app()` — factory, testable config
- `init_app()` — extensions without globals
- `bp.endpoint` — namespaced URLs

Contexts, g & Configuration

app context · request context · current_app · g · config.from_object / from_prefixed_env · SECRET_KEY · .flaskenv

DESCRIPTION

Flask makes `request`, `session`, `current_app` and `g` act like globals by pushing **contexts**: an **app context** (which app is active) and a **request context** (which request). **g** is scratch space for one request, like the current user or a DB connection. Configuration lives in `app.config`, loaded from objects, files or environment variables.

NOTES

- Contexts are pushed automatically during requests and CLI commands
- Outside them, use `with app.app_context():`
- `g` is reset for every request
- `FLASK_` env vars load via `from_prefixed_env()`

UNDER THE HOOD

- Contexts are stored in Python `contextvars`, safe across threads and async
- `request`, `g`, `current_app` are `LocalProxy` objects that look up the active context
- Teardown functions run when the app context is popped

CONTEXTS & CONFIG

```python
from flask import Flask, current_app, g, request

class Default:
    SECRET_KEY = "dev"                    # override in prod!
    ITEMS_PER_PAGE = 20

app = Flask(__name__)
app.config.from_object(Default)
app.config.from_prefixed_env()        # FLASK_SECRET_KEY → SECRET_KEY

@app.before_request
def load_user():
    g.user = users.from_session()          # per-request scratch space

@app.get("/todos")
def todos():
    per_page = current_app.config["ITEMS_PER_PAGE"]
    return repo.for_user(g.user, per_page)

with app.app_context():                    # scripts / shell
    db.create_all()
```

CODE EXPLAINED

- → `from_object(Default)` — load UPPERCASE attributes
- → `from_prefixed_env()` — read `FLASK_*` env vars
- → `g.user` — data for this request only
- → `current_app` — the active app (use in blueprints)
- → `app.app_context()` — run code outside requests

OTHER FRAMEWORK

- FastAPI — dependencies replace `g`; settings via pydantic-settings (sheet 16)
- Django — `request.user` from middleware; `django.conf.settings`

PREFER / AVOID

- Avoid importing `app` in blueprints for config · Prefer `current_app.config`
- Avoid storing request data in module globals · Prefer `g`

CONTEXT OBJECTS

NAME	SCOPE
`current_app`	app context
`g`	app context (one per request)
`request`	request context
`session`	request context

CONFIG LOADERS

METHOD	SOURCE
`from_object(obj)`	class or module
`from_pyfile("x.py")`	Python file
`from_file("x.json", load=json.load)`	JSON/TOML
`from_prefixed_env()`	FLASK_* env vars
`config.update(...)`	dict

⚠ COMMON MISTAKES

- "Working outside of application context" in scripts: wrap in `app.app_context()`
- Shipping `SECRET_KEY = "dev"` to production
- Lowercase config keys: only UPPERCASE names are loaded

✓ TIPS & BEST PRACTICES

- Use `.flaskenv` for `FLASK_APP` / `FLASK_DEBUG` and `.env` for secrets
- Generate keys with `python -c "import secrets; print(secrets.token_hex())"`
- Use `flask shell`; it pushes an app context for you

QUICK SUMMARY

- app + request contexts — make globals safe
- `g` — per-request storage
- `current_app` — active app in blueprints
- `app.config` — objects, files, env vars

Sheet 53

Request Hooks & Error Handling

before_request · after_request · teardown_appcontext · abort(404) · @app.errorhandler · error pages · logging

DESCRIPTION

Request hooks run code around every request: `before_request` (load the user, block requests), `after_request` (add headers), and `teardown_appcontext` (always clean up, even after errors). **`abort()`** raises an HTTP error; **`@app.errorhandler`** turns errors and exceptions into custom pages or JSON.

NOTES

- Returning a value from `before_request` skips the view
- `after_request` must return the response
- `teardown_*` gets the exception (or `None`) and can't change the response
- `app.logger` is a standard `logging.Logger`

HOOKS & HANDLERS

```
from flask import abort, g, request
from werkzeug.exceptions import HTTPException

@app.before_request
def require_login():
    public = {"auth.login", "static"}
    if request.endpoint not in public and not g.get("user"):
        abort(401)

@app.after_request
def security_headers(resp):
    resp.headers["X-Content-Type-Options"] = "nosniff"
    return resp

@app.teardown_appcontext
def close_db(exc):
    if (db := g.pop("db", None)) is not None:
        db.close()

@app.errorhandler(404)
def not_found(e):
    return render_template("404.html"), 404

@app.errorhandler(HTTPException)
def http_error(e):                          # JSON for every HTTP error
    return {"error": e.name, "detail": e.description}, e.code
```

CODE EXPLAINED

- → `before_request` — `abort(401)` stops the request
- → `after_request` — modify and return the response
- → `teardown_appcontext` — always runs; close resources
- → `errorhandler(404)` — custom page + status
- → `errorhandler(HTTPException)` — catch them all

OTHER FRAMEWORK

- FastAPI — middleware + dependencies + `@app.exception_handler` (sheets 19, 27)
- Django — middleware; `handler404` / `handler500` views

PREFER / AVOID

- Avoid `try/except` in every view · Prefer error handlers
- Avoid returning HTML errors from JSON APIs · Prefer a JSON handler

HOOKS

DECORATOR	RUNS
`before_request`	before each view
`after_request`	after view, success
`teardown_request`	end of request, always
`teardown_appcontext`	app context popped, always
`@bp.before_request`	blueprint only

ABORT() CODES

CALL	RESPONSE
`abort(400)`	Bad Request
`abort(401)`	Unauthorized
`abort(403)`	Forbidden
`abort(404)`	Not Found
`abort(409)`	Conflict

⚠ COMMON MISTAKES

- Forgetting `return resp` in `after_request`: request fails
- Expecting `after_request` to run after an unhandled exception: use teardown
- Showing stack traces in production error pages

✓ TIPS & BEST PRACTICES

- Log with `app.logger.exception(...)` inside a 500 handler
- Use `@bp.app_errorhandler` to register app-wide handlers from a blueprint
- Return problem JSON for API blueprints, HTML for pages

QUICK SUMMARY

- `before_request` — gate or prepare
- `after_request` — adjust response
- teardown — always clean up
- `abort()` + `errorhandler` — custom errors

Flask-SQLAlchemy & Flask-Migrate

SQLAlchemy(model_class=Base) · db.session · db.select · db.get_or_404 · flask db init / migrate / upgrade

DESCRIPTION

Flask-SQLAlchemy wires SQLAlchemy into Flask: one `db` object, a session scoped to each request, and helpers like `get_or_404()` and `paginate()`. Models use normal SQLAlchemy 2.0 syntax. **Flask-Migrate** wraps Alembic as `flask db` commands.

NOTES

- `pip install flask-sqlalchemy flask-migrate`
- Config key: `SQLALCHEMY_DATABASE_URI`
- Session is removed automatically at request end
- Relative SQLite paths resolve in `instance/`

MODELS + QUERIES

```python
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase): pass
db = SQLAlchemy(model_class=Base)
migrate = Migrate()
class Todo(db.Model):
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(db.String(100))
    done: Mapped[bool] = mapped_column(default=False)

@bp.get("/todos")
def list_todos():
    page = db.paginate(db.select(Todo), per_page=20)
    return {"items": [t.title for t in page.items]}

@bp.get("/todos/<int:id>")
def show(id):
    todo = db.get_or_404(Todo, id)
    return {"id": todo.id, "title": todo.title}
```

CODE EXPLAINED

- → `SQLAlchemy(model_class=Base)` — 2.0-style models
- → `db.Model` — table name from class name (`todo`)
- → `db.paginate(select)` — page object with `.items`
- → `db.get_or_404()` — fetch or 404
- → `flask db` — Alembic via the Flask CLI

FLASK-MIGRATE

```
flask db init                     # once: migrations/ folder
flask db migrate -m "add todos"   # autogenerate
flask db upgrade                  # apply
```

OTHER FRAMEWORK

- FastAPI — plain SQLAlchemy/SQLModel + yield session dependency (sheets 23, 28)
- Django — built-in ORM + `makemigrations`

PREFER / AVOID

- Avoid legacy `Todo.query.filter_by()` · Prefer `db.session.execute(db.select(...))`
- Avoid `db.create_all()` once you have users · Prefer migrations

DB HELPERS

HELPER	DOES
`db.session.add / commit`	write
`db.session.execute(q)`	run query
`db.session.scalars(q).all()`	list of models
`db.get_or_404(M, id)`	by PK or 404
`db.first_or_404(q)`	first or 404
`db.paginate(q)`	pagination

FLASK DB COMMANDS

COMMAND	DOES
`init`	create migrations/
`migrate -m`	autogenerate
`upgrade`	apply
`downgrade`	revert one
`history / current`	inspect

⚠ COMMON MISTAKES

- Forgetting `migrate.init_app(app, db)` in the factory: `flask db` can't find models
- Models not imported anywhere: autogenerate sees no tables
- Long-lived objects used after the request's session closed

✓ TIPS & BEST PRACTICES

- Import models in `create_app()` so migrations detect them
- Turn on `SQLALCHEMY_ECHO = True` while debugging queries
- Review migration files before `upgrade`

QUICK SUMMARY

- `db = SQLAlchemy()` — Flask-integrated ORM
- `db.Model` + `Mapped[...]` — models
- `get_or_404` / `paginate` — view helpers
- `flask db` — Alembic migrations

Sessions & Flask-Login

DESCRIPTION

Flask's **`session`** stores small data in a **signed cookie**: users can read it but can't change it without the SECRET_KEY. **Flask-Login** builds user logins on top: it remembers the user ID in the session, loads the user each request into **`current_user`**, and protects views with **`@login_required`**.

NOTES

- Session cookie is signed, **not encrypted**: no secrets inside
- Flask-Login doesn't hash passwords or render forms
- `user_loader` turns the stored ID back into a user
- `remember=True` sets a long-lived remember cookie

LOGIN FLOW

```python
from flask_login import (LoginManager, UserMixin, current_user,
                         login_required, login_user)
from werkzeug.security import check_password_hash as check_pw

login_manager = LoginManager()
login_manager.login_view = "auth.login"     # redirect target
class User(UserMixin, db.Model): ...         # is_authenticated, get_id

@login_manager.user_loader
def load_user(user_id: str):
    return db.session.get(User, int(user_id))

@bp.post("/login")
def login():
    f = request.form
    user = db.session.scalar(db.select(User).filter_by(email=f["email"]))
    if user and check_pw(user.password_hash, f["password"]):
        login_user(user, remember=True)
        return redirect(url_for("todos.index"))
    flash("Invalid credentials")
    return redirect(url_for("auth.login"))

@bp.get("/dashboard")
@login_required
def dashboard():
    session["visits"] = session.get("visits", 0) + 1
    return f"Hi {current_user.email} (#{session['visits']})"
```

CODE EXPLAINED

- → `login_view` — where anonymous users get sent
- → `UserMixin` — default user properties
- → `user_loader` — ID from session → user
- → `login_user()` / `logout_user()` — start/end
- → `session[...]` — small signed per-user data

OTHER FRAMEWORK

- FastAPI — OAuth2 + JWT dependencies (sheets 32–33)
- Django — built-in `login()`, `@login_required`, `request.user`

PREFER / AVOID

- Avoid redirecting to `next` blindly · Prefer checking it is a local URL
- Avoid storing user objects in `session` · Prefer the user ID only

FLASK-LOGIN API

NAME	DOES
`login_user(u)`	log in
`logout_user()`	log out
`current_user`	user or anonymous
`@login_required`	protect view
`current_user.is_authenticated`	check in templates

SESSION COOKIE CONFIG

KEY	SET TO
`SECRET_KEY`	long random value
`SESSION_COOKIE_SECURE`	True (HTTPS)
`SESSION_COOKIE_HTTPONLY`	True (default)
`SESSION_COOKIE_SAMESITE`	"Lax"
`PERMANENT_SESSION_LIFETIME`	timedelta(days=7)

⚠ COMMON MISTAKES

- Open redirect via `?next=https://evil.com`
- Rotating `SECRET_KEY` and wondering why everyone is logged out
- Putting large data in `session`: browsers cap cookies near 4 KB

✓ TIPS & BEST PRACTICES

- Hash with `generate_password_hash()` (scrypt by default)
- Use Flask-Session for server-side sessions (Redis) if you need more space
- Add `@login_required` at blueprint level with `before_request`

QUICK SUMMARY

- `session` — signed cookie storage
- `LoginManager` + `user_loader` — load users
- `@login_required` — protect views
- `current_user` — the logged-in user

REST APIs & Async in Flask

JSON APIs · MethodView · flask-smorest / APIFlask · async def views · flask-cors · async limits on WSGI

DESCRIPTION

Flask builds JSON APIs well: return dicts, use **`MethodView`** for one class per resource, and add **APIFlask** or **flask-smorest** if you want validation and OpenAPI docs like FastAPI. Flask also accepts **`async def`** views, but on WSGI each async request still occupies a worker, so it helps only for concurrent calls inside one request.

NOTES

- `pip install "flask[async]"` enables async views
- Each async view runs its own event loop in a thread
- Use Quart if you want a fully async Flask-like framework
- `flask-cors` adds CORS headers

METHODVIEW + ASYNC VIEW

```python
import asyncio, httpx
from flask.views import MethodView
from flask_cors import CORS

class TodoAPI(MethodView):
    def get(self, todo_id):
        return db.get_or_404(Todo, todo_id).to_dict()
    def patch(self, todo_id):
        todo = db.get_or_404(Todo, todo_id)
        todo.update(request.get_json()); db.session.commit()
        return todo.to_dict()
    def delete(self, todo_id):
        db.session.delete(db.get_or_404(Todo, todo_id))
        db.session.commit()
        return "", 204

app.add_url_rule("/todos/<int:todo_id>",
                 view_func=TodoAPI.as_view("todo"))
CORS(app, origins=["https://app.example.com"])

@app.get("/dashboard-data")
async def dashboard_data():
    async with httpx.AsyncClient() as c:
        a, b = await asyncio.gather(c.get(URL_A), c.get(URL_B))
    return {"weather": a.json(), "news": b.json()}
```

CODE EXPLAINED

- → `MethodView` — methods map to HTTP verbs
- → `as_view("todo")` — endpoint name
- → `return "", 204` — empty success
- → `CORS(app, origins=)` — allow a frontend origin
- → async view — two calls at once, inside one request

OTHER FRAMEWORK

- FastAPI — validation, docs and real async built in
- Django — DRF `APIView` / `ViewSet` + routers

PREFER / AVOID

- Avoid hand-rolled validation for big APIs · Prefer APIFlask, flask-smorest or Pydantic
- Avoid async views expecting more throughput · Prefer an ASGI framework for that

API ADD-ONS

LIBRARY	ADDS
APIFlask	schemas, OpenAPI, docs UI
flask-smorest	Marshmallow + OpenAPI
Flask-Pydantic	Pydantic validation decorator
flask-cors	CORS
Flask-Limiter	rate limiting

ASYNC OPTIONS

OPTION	RESULT
async def view (WSGI)	concurrency inside one request
Quart	Flask API on ASGI
Gunicorn gevent worker	many concurrent sync requests
FastAPI	native async

⚠ COMMON MISTAKES

- Expecting async views to handle more requests per worker on WSGI
- `CORS(app)` with defaults in production: allows every origin
- Forgetting `flask[async]` and getting a RuntimeError

✓ TIPS & BEST PRACTICES

- Use `jsonify` only for lists in very old Flask; modern Flask accepts lists
- Version API blueprints: `url_prefix="/api/v1"`
- Consider FastAPI for new API-only services

QUICK SUMMARY

- `MethodView` — class per resource
- APIFlask / smorest — validation + OpenAPI
- `async def` views — concurrency within a request
- `flask-cors` — cross-origin access

Testing & Deploying Flask

app.test_client() · test_cli_runner() · TESTING=True · pytest fixtures · Gunicorn · Waitress · ProxyFix

DESCRIPTION

Test Flask apps with the built-in **test client** and **pytest fixtures** that build the app through `create_app()` with test config. Deploy with a production **WSGI server**: **Gunicorn** on Linux, **Waitress** on Windows, behind Nginx. Add **`ProxyFix`** so Flask trusts the proxy's forwarded headers.

NOTES

- `TESTING=True` propagates exceptions to the test
- Disable CSRF in tests: `WTF_CSRF_ENABLED=False`
- Gunicorn workers ≈ 2 × cores + 1 for sync workers
- Never deploy with `flask run`

TESTS/CONFTEST.PY + TEST

```python
import pytest
from myapp import create_app
from myapp.extensions import db

@pytest.fixture
def app():
    app = create_app({"TESTING": True, "WTF_CSRF_ENABLED": False,
                      "SQLALCHEMY_DATABASE_URI": "sqlite:///:memory:"})
    with app.app_context():
        db.create_all()
        yield app

@pytest.fixture
def client(app):
    return app.test_client()

def test_create_todo(client):
    r = client.post("/todos", json={"title": "Buy milk"})
    assert r.status_code == 201
    assert r.get_json()["title"] == "Buy milk"
```

CODE EXPLAINED

- → `create_app({...})` — test config per run
- → `app.app_context()` — DB setup inside context
- → `app.test_client()` — fake requests
- → `r.get_json()` — parsed JSON
- → `"myapp:create_app()"` — Gunicorn calls the factory

OTHER FRAMEWORK

- FastAPI — `TestClient` + Uvicorn workers (sheets 40, 45)
- Django — `TestCase` client; Gunicorn + `collectstatic`

PRODUCTION

```
pip install gunicorn
gunicorn -w 4 -b 127.0.0.1:8000 "myapp:create_app()"
# app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1)
```

PREFER / AVOID

- Avoid testing against the dev database · Prefer an isolated test DB
- Avoid `app.run(debug=True)` on servers · Prefer Gunicorn/Waitress

TEST CLIENT

CALL	DOES
`client.get(url)`	GET
`client.post(url, json=)`	JSON POST
`client.post(url, data=)`	form POST
`follow_redirects=True`	follow 302s
`with client.session_transaction() as s:`	edit session

WSGI SERVERS

SERVER	NOTES
`Gunicorn`	Linux, pre-fork workers
`Waitress`	pure Python, Windows OK
`uWSGI`	powerful, maintenance mode
`Gunicorn + gevent`	many idle connections

⚠ COMMON MISTAKES

- Tests sharing state through a module-level app
- Forgetting `ProxyFix`: `url_for(_external=True)` builds `http://` links
- Running Gunicorn as root on port 80 instead of behind Nginx

✓ TIPS & BEST PRACTICES

- Use `app.test_cli_runner()` to test custom `flask` commands
- Run Gunicorn under systemd (see the Ubuntu set)
- Serve `static/` from Nginx in production

QUICK SUMMARY

- `create_app(test_config)` — isolated tests
- `test_client()` — fake requests
- Gunicorn / Waitress — production servers
- `ProxyFix` — trust proxy headers

FastAPI vs Flask vs Django

DESCRIPTION

All three are solid, widely used Python web frameworks; they differ in **what's built in**. **FastAPI** is best for typed JSON APIs with automatic validation and docs. **Flask** is a small, flexible core for small apps, server-rendered pages and teams that want to pick each piece. **Django** is batteries-included: ORM, admin, auth and forms for full web apps.

NOTES

- All three can serve HTML and JSON
- Django and Flask both support async views; FastAPI is async-native
- Mixing is common: Django admin + FastAPI service
- Pick the one your team will maintain well

SAME ENDPOINT, THREE WAYS

```python
# FastAPI
@app.get("/todos/{todo_id}")
def get_todo(todo_id: int) -> TodoOut:
    return repo.get(todo_id)                # validated + documented

# Flask
@app.get("/todos/<int:todo_id>")
def get_todo(todo_id):
    return repo.get(todo_id).to_dict()      # you shape the JSON

# Django (urls.py + views.py)
path("todos/<int:todo_id>/", views.get_todo)
def get_todo(request, todo_id):
    todo = get_object_or_404(Todo, pk=todo_id)
    return JsonResponse(model_to_dict(todo))
```

CODE EXPLAINED

- → FastAPI — types do the parsing, validation and docs
- → Flask — converter in the URL; manual serialization
- → Django — URLconf + view + ORM helper
- → Same result; different amounts of built-in help

PREFER / AVOID

- Avoid choosing by benchmarks alone · Prefer fit with your app's needs
- Avoid rewriting a working app to switch · Prefer new services in the new framework

FEATURE COMPARISON

FEATURE	FASTAPI	FLASK	DJANGO
Interface	ASGI	WSGI	WSGI + ASGI
Validation	built in	extension	forms / DRF
API docs	automatic	extension	DRF + spectacular
ORM	bring your own	Flask-SQLAlchemy	built in
Admin UI	no (SQLAdmin)	Flask-Admin	built in
Templates	Jinja2	Jinja2	DTL / Jinja2

PICK WHEN

CHOOSE	IF YOU NEED
FastAPI	typed JSON APIs, async I/O, OpenAPI clients
Flask	small app, freedom, simple HTML
Django	full site, admin, auth, CMS-like data
Flask → FastAPI	APIs outgrowing manual validation

⚠ COMMON MISTAKES

- Rebuilding Django's admin and auth in FastAPI for a CRUD back office
- Choosing FastAPI for async, then using only sync libraries
- Assuming Flask can't scale; plenty of large sites run it

✓ TIPS & BEST PRACTICES

- Migrating Flask → FastAPI: port blueprints to routers one at a time
- Mount a Flask app inside FastAPI with `WSGIMiddleware` during migration
- Keep business logic framework-free so switching is cheap

QUICK SUMMARY

- FastAPI — typed APIs, docs, async
- Flask — small core, choose extensions
- Django — batteries included
- mix and migrate gradually

Quick Reference — FastAPI

PATH OPERATIONS

SYNTAX	DOES	#
`@app.get("/x/{id}")`	GET route	05
`@app.post(..., status_code=201)`	create	05
`@router.get(...)`	router route	25
`app.include_router(r)`	mount router	25
`@app.websocket("/ws")`	websocket	38
`app.mount("/static", ...)`	sub-app	42

PARAMETERS

SYNTAX	SOURCE	#
`id: int (in path)`	path	06
`q: str | None = None`	query	07
`item: Model`	JSON body	08
`Annotated[int, Query(le=100)]`	validated query	09
`Annotated[str, Header()]`	header	10
`Annotated[str, Form()]`	form field	10
`file: UploadFile`	upload	10

DEPENDENCIES & SECURITY

SYNTAX	DOES	#
`Annotated[T, Depends(fn)]`	inject	21
`yield in dependency`	setup/teardown	23
`dependencies=[Depends(f)]`	run, ignore result	24
`app.dependency_overrides[f]`	test swap	41
`OAuth2PasswordBearer(tokenUrl=)`	Bearer token	32
`Security(f, scopes=[...])`	scopes	35

RESPONSES & ERRORS

SYNTAX	DOES	#
`-> OutModel`	validate + filter	15
`response: Response`	headers/cookies	18
`raise HTTPException(404)`	error	19
`@app.exception_handler(E)`	custom error	19
`StreamingResponse(gen())`	stream	39
`BackgroundTasks.add_task`	after response	37

PYDANTIC

SYNTAX	DOES	#
`Field(gt=0, max_length=50)`	constraints	12
`@field_validator("x")`	field rule	13
`model_dump(exclude_unset=True)`	partial dict	11
`ConfigDict(extra="forbid")`	strict keys	16
`BaseSettings`	env config	16

CLI & RUN

COMMAND	DOES	#
`fastapi dev main.py`	dev + reload	02
`fastapi run --workers 4`	production	45
`uvicorn main:app --reload`	server direct	02
`alembic upgrade head`	migrate	31
`pytest -q`	tests	40

⚠ COMMON MISTAKES

- `async def` with blocking calls freezes the worker (36)
- Param route registered before a fixed route steals it (05)
- Returning DB models without an Out schema leaks fields (15)

✓ TIPS & BEST PRACTICES

- Check `/docs` after every change
- Put shared `Annotated` aliases in `deps.py`
- Override dependencies in tests, don't patch

QUICK SUMMARY

- type hints — parse, validate, document
- `Depends` — share logic and resources
- Out models — control responses
- `fastapi run` — production

Quick Reference — Flask

ROUTING

SYNTAX	DOES	#
`@app.get("/x/<int:id>")`	route + converter	47
`@app.route(..., methods=[...])`	multi-method	47
`url_for("bp.view", id=1)`	build URL	47
`redirect(url)`	redirect	47
`abort(404)`	error	53
`Blueprint(..., url_prefix=)`	route group	51

REQUEST

SYNTAX	GIVES	#
`request.args.get(k, type=int)`	query value	48
`request.args.getlist(k)`	repeated	48
`request.form[k]`	form field	48
`request.get_json()`	JSON body	48
`request.files["f"]`	upload	48
`request.headers / cookies`	metadata	48

RESPONSES & TEMPLATES

SYNTAX	DOES	#
`return {...}`	JSON	48
`return body, 201, headers`	status + headers	48
`make_response(...)`	editable response	48
`render_template("x.html", **ctx)`	HTML	49
`{% extends %} / {% block %}`	layouts	49
`flash(msg)`	one-time message	50

APP & CONTEXT

SYNTAX	DOES	#
`create_app(config)`	factory	51
`ext.init_app(app)`	bind extension	51
`current_app.config[k]`	settings	52
`g.user`	per-request data	52
`@app.before_request`	hook	53
`@app.errorhandler(404)`	error page	53

EXTENSIONS

EXTENSION	KEY API	#
`Flask-SQLAlchemy`	db.get_or_404, db.paginate	54
`Flask-Migrate`	flask db migrate	54
`Flask-WTF`	validate_on_submit()	50
`Flask-Login`	@login_required	55
`Flask-CORS`	CORS(app)	56

CLI

COMMAND	DOES	#
`flask --app app run --debug`	dev server	46
`flask routes`	list routes	47
`flask shell`	shell with context	52
`flask db upgrade`	migrate	54
`gunicorn -w 4 "pkg:create_app()"`	production	57

⚠ COMMON MISTAKES

- `request` used outside a request context (52)
- `after_request` without returning the response (53)
- Deploying with `flask run` instead of Gunicorn (57)

✓ TIPS & BEST PRACTICES

- Start with the app factory and blueprints
- Use `url_for` everywhere, even in templates
- Keep secrets in env vars loaded by `from_prefixed_env()`

QUICK SUMMARY

- routes + `url_for` — URLs
- `request` — inputs; tuples — responses
- factory + blueprints — structure
- extensions — DB, forms, login